UNIVERSITY

FACULTY OF INFORMATION TECHNOLOGY

BACHELOR'S GRADUATION THESIS

Budget-Aware Large Language Model Calls for MPaGE-Based Multi-Objective Heuristic Design

Proposed Framework: Budget-aware MPaGE-style heuristic design

Major:	Computer Science
Research area:	Generative AI and multi-objective combinatorial optimization
Typesetting system:	XeLaTeX

June 23, 2026

Abstract

This thesis studies how a finite budget of large language model (LLM) calls can be allocated more effectively in automatic heuristic design for multi-objective combinatorial optimization. The study builds on MPaGE, a framework that combines LLM-based program generation, SEMO-style multi-objective evolution, Pareto Front Grid selection, and semantic clustering of generated heuristics. The proposed framework replaces the largely fixed scheduling policy with a two-level budgeted multi-armed bandit controller: the first level selects the variation operator, and the second level selects the semantic cluster to exploit in the current generation. The implementation also introduces explicit budget accounting, Page–Hinkley drift detection, bounded reward computation, reward alignment mechanisms, and the area under the budget curve (AUBC) as a budget-efficiency metric.

This thesis synthesizes the theoretical background, system architecture, implementation details, and experimental evidence available in the implementation and result artifacts. The current result set contains 320 runs over four setups, four benchmark problem families, four LLM-call budgets, and five random seeds. The analyzed setups include three parallel quality-signal variants, namely Final-HV reward, Dense reward, and Hybrid reward, together with the budget-wrapped MPaGE-orig baseline. The empirical evidence indicates that the BMAB-style variants substantially improve AUBC over MPaGE-orig. Among the three quality-signal variants, Final-HV reward provides the strongest final-HV ranking, while Hybrid reward provides the strongest mean normalized AUBC score by a small margin.

Contents

List of Figures

List of Tables

Chapter 1

Introduction

1.1 Research Context

Multi-objective combinatorial optimization is a central problem class in computer science. It appears in routing, scheduling, resource allocation, logistics, packing, and system design. Unlike single-objective optimization, the goal is not to identify one globally best solution under a scalar criterion. Instead, the algorithm must approximate a set of Pareto-optimal trade-offs, where improving one objective may necessarily degrade another. This requirement makes the design of effective search heuristics substantially more difficult, particularly when the feasible space is discrete and grows combinatorially with the problem size.

In many multi-objective combinatorial optimization problems (MOCOPs), the effectiveness of a heuristic depends strongly on the structure of the target task. A neighborhood rule that works well for a bi-objective traveling salesman problem may not transfer directly to a capacitated vehicle routing problem or a bi-objective knapsack problem. This task dependence motivates automatic heuristic design: instead of manually specifying every search rule, a system can generate candidate heuristic programs, execute them in a controlled evaluator, and retain the programs that produce stronger Pareto-front approximations.

Recent work has explored the use of large language models (LLMs) as program generators for this purpose. The MPaGE framework proposes Pareto-Grid-Guided LLMs for heuristic design in multi-objective combinatorial optimization [1]. Its main mechanism combines LLM-generated heuristic code, SEMO-style multi-objective evolution, Pareto Front Grid (PFG) selection, and LLM-based semantic clustering. This thesis builds on that foundation but addresses a narrower resource-allocation question: when the number of LLM calls is limited, how should the system allocate those calls to obtain useful heuristic populations earlier and more reliably?

This question is important because LLM-based heuristic design differs from ordinary evolutionary search in both cost structure and failure behavior. In a classical evolutionary

algorithm, generating one additional mutation can often be treated as a relatively cheap operation compared with evaluating the population. In an LLM-driven system, however, each candidate heuristic requires a model call, prompt construction, response parsing, code extraction, execution in a sandboxed evaluator, and population update. The generated program may be syntactically valid but semantically useless, or it may fail because it violates a task-specific contract. Therefore, the design of the outer search policy is not only an algorithmic preference; it directly determines how a scarce generative resource is converted into validated executable heuristics.

The study also has broader research relevance within generative AI. Many applications of LLMs use a generate–evaluate–select loop, but the scheduling of generation calls is often treated as a fixed engineering detail. In contrast, this thesis studies generation as a sequential decision process. The LLM is not fine-tuned, and it is not used as an autonomous multi-agent planner. Instead, it acts as a stochastic program generator embedded inside an optimization loop whose state, rewards, and budget consumption are explicitly measured. This framing makes the proposed framework a concrete example of how generative models can be combined with classical decision-theoretic control.

1.2 Problem Statement

The available implementation documentation identifies budget allocation as a practical limitation of the baseline MPaGE workflow. MPaGE already provides a productive mechanism for generating and evolving heuristic programs, but its scheduling policy is comparatively static. In each generation, the system selects heuristic elites, clusters them semantically, applies mutation or crossover prompts, evaluates the resulting code, and updates the heuristic population. This process is effective as a heuristic-design loop, but it does not explicitly learn which operator or semantic region is currently yielding the most valuable improvement per LLM call.

This limitation matters because an LLM call is not a free computational step. It consumes API budget, wall-clock time, and experimental opportunity. Moreover, not every generated program is valid: some candidates fail to parse, violate the expected function signature, time out, or return infeasible outputs during evaluation. Therefore, the outer loop of LLM-driven heuristic design must be treated not only as an optimization process over generated programs, but also as a sequential decision problem under a hard budget.

The central problem studied in this thesis is therefore the following:

> How can an MPaGE-style LLM heuristic-design system allocate a fixed budget of generation and clustering calls so that the managed heuristic population achieves high hypervolume earlier, while preserving the final quality and diver-

sity of the heuristic Pareto front?

The problem is deliberately formulated at the level of *heuristic-population search*. Each generated program is evaluated by an inner solver on fixed benchmark instances, but the proposed method does not directly optimize tours, vehicle routes, or knapsack vectors. Instead, it optimizes a population of heuristic programs that can produce such solutions. This creates a two-level structure: the inner level measures how well a heuristic solves a benchmark problem, while the outer level decides which heuristic programs should survive and which future generation calls should be made. Confusing these two levels can lead to incorrect interpretations of the results, so the thesis consistently distinguishes inner solution-level quality from outer heuristic-population quality.

1.3 Research Questions

The thesis is organized around four research questions.

1. **Budget efficiency.** Does a budget-aware scheduling policy improve the area under the outer hypervolume-vs-budget curve compared with the budget-wrapped MPaGE baseline?

2. **Terminal quality.** Does improving budget efficiency also preserve or improve final heuristic-population hypervolume, or does it merely move progress earlier without changing the final population?

3. **Reward design.** How do immediate dense HVI reward, final managed-population HV reward, and their hybrid differ in practice when all other finalized implementation components are held fixed?

These questions are narrower than a complete re-evaluation of MPaGE across every table in the original paper. They focus on the budgeted outer search process implemented in this thesis. This choice is intentional. The most distinctive contribution of the study is not a new benchmark task or a new LLM prompt family, but a controller that decides how to spend a fixed call budget across operators and semantic regions.

1.4 Research Motivation

The motivation for this thesis is supported by three observations found directly in the implementation and experimental artifacts.

First, the implementation treats LLM calls as a limited resource. The system includes a hard budget tracker that records budget consumption through explicit charging operations. Both the proposed method and the MPaGE baseline wrapper receive the same budget argument so that methods can be compared under a common call budget.

Second, generated heuristics can be invalid or uninformative. The secure evaluation layer executes generated programs in a separate process and may return a null score when the code is syntactically invalid, semantically incompatible with the task, or exceeds the allowed runtime. This makes each LLM call a risky investment rather than a guaranteed source of improvement.

Third, terminal hypervolume alone is insufficient for evaluating a budget-constrained LLM search process. As defined in the metric formulation used throughout this thesis, AUBC is the area under the hypervolume-vs-budget curve and measures how quickly a method converts consumed budget into heuristic-population quality. Two methods may reach similar final hypervolume, but the method that reaches high-quality fronts earlier is more useful when LLM calls are expensive or when a run may be stopped before exhausting a large budget.

These observations motivate a controller that learns from the outcomes of previous generation calls. If crossover in one generation produces valid and diverse children, it should become more attractive; if a semantic cluster becomes saturated and no longer improves the population, it should receive fewer calls. Conversely, unexplored operators or clusters should not be abandoned too early, because the apparent lack of reward may be caused by limited evidence. This is precisely the exploration–exploitation tension studied in multi-armed bandits. The novelty in this thesis is the adaptation of that idea to the MPaGE heuristic-design loop, where arms correspond to variation operators and semantic cluster–operator combinations rather than ordinary solution-space moves.

The motivation is also practical for experimental research. LLM calls can make large grids of experiments expensive and time-consuming. A method that reaches a competitive heuristic population using fewer useful calls can make iterative research faster, especially during ablation studies, prompt debugging, and preliminary task exploration. AUBC is therefore not merely a secondary metric; it is a way of measuring whether the system makes meaningful progress throughout the budget horizon.

1.5 Research Objectives

The thesis has five concrete objectives.

1. Analyze the MPaGE baseline and identify the retained components, including the task templates, prompt families, semantic clustering, PFG selection, and secure

evaluation infrastructure.

2. Present the proposed BMAB-LLM method as a budget-aware extension of MPaGE. The discussion covers hard budget accounting, operator-level bandit scheduling, cluster-level bandit scheduling, Page–Hinkley drift detection, final-HV-oriented reward computation, budget-aware exploration annealing, and final population flushing.

3. Explain the system implementation, benchmark evaluators, experimental harness, and reproducibility artifacts at a level sufficient for reproducing the experimental protocol.

4. Analyze the available experimental evidence summarized in the thesis tables and figures, focusing on AUBC, final hypervolume, normalized scores, mean ranks, pairwise task–budget wins, and budget-curve behavior.

5. Draw evidence-controlled conclusions. Claims are made only when they are supported by source code, documentation, or experimental artifacts. When information is absent, the thesis explicitly states that no information was found in the source code or documentation.

1.6 Research Scope

The empirical scope of this thesis is the analyzed comparison matrix represented by the thesis tables and figures. The current complete matrix contains four thesis-facing setups: three BMAB-style quality-signal variants, Final-HV reward, Dense reward, and Hybrid reward, together with MPaGE-orig as the original workflow wrapped under the same budget-recording protocol. The evaluated tasks are Bi-TSP, Tri-TSP, Bi-CVRP, and Bi-KP. The LLM-call budgets are 25, 50, 100, and 200. Each task–budget–setup cell is run with five seeds, from 2025 to 2029, according to the experimental matrix described in Chapter 5.

Several additional ablation variants are implemented in the codebase, including `no_budget_anneal`, `no_ph`, `no_diversity`, `op_only`, and `cluster_only`. These variants are important for future causal analysis, but they are not all present as complete result matrices in the current experimental directory. Consequently, Chapter 5 analyzes the four complete thesis-facing setups and treats the remaining unexecuted or incomplete ablations as future work rather than as established empirical findings.

The thesis does not re-evaluate the benchmark tables reported in the original MPaGE paper. The available result files measure outer heuristic-population hypervolume and AUBC under the experimental framework used in this study. They should therefore be interpreted

as evidence about budget-aware heuristic-population search, not as a direct reproduction of all normalized inner-solver metrics in the original paper.

The implementation does not provide evidence of LLM fine-tuning, LoRA, PEFT, retrieval-augmented generation, diffusion models, GANs, knowledge distillation, or multi-agent autonomous planning. These topics are therefore not claimed as contributions of this thesis.

1.7 Research Methodology

The thesis follows an evidence-based implementation analysis methodology. First, README files, design documents, implementation modules, evaluator definitions, scripts, result files, and generated figures were inspected to reconstruct the research problem and implementation architecture. Second, the mathematical and algorithmic foundations were organized around multi-objective optimization, hypervolume, LLM-based program synthesis, adaptive operator selection, budgeted multi-armed bandits, and online drift detection. Third, the result CSV and JSON files were analyzed using the metrics defined for this study, particularly AUBC and final hypervolume.

The primary empirical metric is AUBC because it directly evaluates budget efficiency. Final hypervolume is used as a complementary terminal-quality metric. Chapter 5 reports normalized scores, mean ranks, cell-level pairwise wins, and budget curves generated from the result artifacts.

1.8 Thesis Organization

Chapter 2 reviews the theoretical background and related work: multi-objective combinatorial optimization, Pareto dominance, hypervolume, MPaGE, LLM-based program generation, multi-armed bandits, budgeted bandits, Page–Hinkley drift detection, and AUBC. Chapter 3 presents the proposed BMAB-LLM method in detail, including the problem formulation, architecture, reward computation, bandit scheduler, and complete workflow. Chapter 4 describes the system implementation, benchmark evaluators, experimental harness, output artifacts, and reproducibility constraints. Chapter 5 analyzes the available experimental results and discusses the observed trends, limitations, and practical significance. Chapter 6 concludes the thesis and outlines future research directions.

Chapter 2

Background and Related Work

2.1 Multi-Objective Combinatorial Optimization

A multi-objective combinatorial optimization problem (MOCOP) can be written as

$$\min_{x \in \mathcal{X}} F(x) = \min_{x \in \mathcal{X}} \left(f_1(x), f_2(x), \ldots, f_m(x) \right), \tag{2.1}$$

where $\mathcal{X}$ is a discrete feasible set and m is the number of objectives. For two feasible solutions x and y, x Pareto-dominates y if $f_i(x) \leq f_i(y)$ for all objectives and $f_j(x) < f_j(y)$ for at least one objective. The set of non-dominated solutions forms a Pareto approximation, which represents different trade-offs rather than a single optimum.

The four benchmark families analyzed in this thesis instantiate this general setting. Bi-TSP and Tri-TSP evaluate tours under two or three distance objectives. Bi-CVRP evaluates route sets under distance-related objectives while enforcing vehicle-capacity constraints. Bi-KP evaluates binary item-selection vectors under two value objectives and a capacity constraint. In all cases, the inner solver maintains an archive of non-dominated candidate solutions and updates it when a generated heuristic proposes a feasible and non-dominated neighbor.

An important distinction in this study is that the LLM-generated program is not itself a final solution to the combinatorial problem. The generated code implements a function such as `select_neighbor`, which acts as a neighborhood operator inside an evaluator. The evaluator repeatedly calls this operator, updates an archive, and then computes the hypervolume of the resulting archive. This pattern is used consistently by the benchmark evaluators analyzed in this thesis.

2.2 Benchmark Problem Families

The benchmark problems used in the experimental framework are deliberately diverse. They are all multi-objective and combinatorial, but they stress different properties of a generated heuristic. This diversity is important because a method that only performs well on one family may simply exploit a narrow representation bias rather than discovering generally useful heuristic-design behavior.

Bi-TSP and Tri-TSP are routing problems over complete graphs. A candidate solution is a permutation of cities, and a neighbor operator typically modifies the tour by swapping, reversing, inserting, or otherwise rearranging city positions. In Bi-TSP, the objective vector has two tour-cost components. In Tri-TSP, the same representation is evaluated under three distance matrices. The tri-objective version is harder from the perspective of the inner archive because a larger objective dimension generally increases the number of mutually non-dominated solutions. As a result, preserving useful diversity in the generated neighbor operator becomes more important.

Bi-CVRP introduces capacity-constrained route construction. A solution is not only an ordering of locations but a set of vehicle routes that must respect a capacity limit. A generated heuristic must therefore modify a structured route set while avoiding infeasible route loads. Compared with TSP, this task gives more opportunities for invalid or low-quality local moves, because changing the position of one customer can affect route feasibility and route balance.

Bi-KP uses a binary decision vector representing selected items. The objectives are value-related and the capacity constraint restricts the feasible subset of items. This problem has a different neighborhood geometry from routing: local moves often correspond to adding, removing, or exchanging selected items. A heuristic that performs well on routing tasks may not transfer directly to Bi-KP because permutation-based intuitions are less useful. Including Bi-KP therefore tests whether the outer heuristic-design process can adapt across representations.

All four tasks evaluate a generated heuristic on multiple fixed problem instances and aggregate its behavior into a two-dimensional outer score. The first score component is the negated mean inner hypervolume, and the second is mean runtime. This convention turns heuristic evaluation into a minimization problem: lower negated hypervolume is better, and lower runtime is better. The convention is simple but important, because it allows the same outer population manager to compare heuristics even when the inner problem has different objective dimensionalities.

2.3 Heuristic Design as a Two-Level Optimization Problem

The proposed framework can be understood as a two-level optimization process. At the inner level, a generated heuristic h is applied to a benchmark instance ξ to produce a set of candidate solutions:

$$A(h, \xi) = \text{Archive}\,(h, \xi)\,, \tag{2.2}$$

where $A(h, \xi)$ denotes the non-dominated archive returned by the evaluator after repeatedly applying the heuristic. The evaluator then computes an inner quality value, usually by hypervolume:

$$q(h, \xi) = HV_{\text{inner}}\,(A(h, \xi))\,. \tag{2.3}$$

Across a set of benchmark instances Ξ, the heuristic obtains an aggregated score:

$$s(h) = \left(-\frac{1}{|\Xi|} \sum_{\xi \in \Xi} q(h, \xi), \frac{1}{|\Xi|} \sum_{\xi \in \Xi} \tau(h, \xi) \right), \tag{2.4}$$

where $\tau(h, \xi)$ is the runtime of the inner evaluation. At the outer level, the algorithm maintains a population $\mathcal{H}$ of generated heuristics and seeks a high-quality Pareto set in the score space:

$$\mathcal{H}^\star \approx \text{ND}\{s(h) : h \in \mathcal{H}\}, \tag{2.5}$$

where ND denotes the non-dominated subset. This formulation clarifies why the thesis reports outer hypervolume and AUBC. The experiments do not directly compare individual tours or knapsack selections. They compare search processes that produce populations of heuristic programs.

This two-level view also explains the role of runtime. A heuristic that produces a strong inner archive but is consistently slow may be dominated by a slightly weaker but much faster heuristic. This is appropriate because automatic heuristic design should discover programs that are both effective and usable. The outer population therefore represents a trade-off between solution quality and computational cost.

2.4 Evolutionary Multi-Objective Optimization

Classical evolutionary multi-objective optimization provides the algorithmic context for MPaGE and BMAB-LLM. NSGA-II uses non-dominated sorting and crowding distance to maintain both convergence and diversity in a population [2]. MOEA/D decomposes a

multi-objective problem into scalar subproblems and optimizes them cooperatively [3]. SEMO-style algorithms use mutation and monotone archive updates to analyze and solve multi-objective pseudo-Boolean and combinatorial problems [4].

MPaGE inherits the archive-based and Pareto-based view of these methods, but changes the search object. Instead of evolving solution vectors directly, it evolves heuristic programs. A heuristic program is evaluated by running it inside a problem-specific solver and measuring the quality and runtime of the resulting Pareto archive. Thus, the outer search problem is itself multi-objective: it seeks heuristics that produce high inner-solver hypervolume while remaining computationally efficient.

This change of search object has methodological consequences. In a conventional evolutionary algorithm, variation operators act on solutions whose representation is known in advance. In MPaGE and BMAB-LLM, the variation operators are prompt templates applied to source code. Mutation may ask the LLM to improve or diversify one parent heuristic, while crossover may ask it to combine ideas from two parent heuristics. The offspring is therefore a program, not a vector. Evaluation is also more expensive and more failure-prone, because the generated program must first satisfy the task interface before it can be scored.

The use of a Pareto population at the outer level remains essential. If the system selected only the single heuristic with the largest inner hypervolume, it could discard faster heuristics that are practically useful or semantically diverse heuristics that later serve as good parents. Maintaining a front of heuristic programs gives the LLM richer context for future generation and reduces the risk of collapsing to one local programming pattern.

2.5 Hypervolume as a Quality Indicator

The hypervolume indicator measures the volume of the region dominated by a set of objective vectors and bounded by a reference point [5, 6]. For minimization, larger hypervolume generally indicates that the approximation set is closer to the Pareto-optimal region and covers a broader range of trade-offs. Hypervolume is widely used because it combines convergence and diversity into a single scalar indicator, although its computational cost grows with the number of objectives.

This thesis uses hypervolume at two different levels. The *inner hypervolume* is computed over solution archives produced by a generated heuristic on a benchmark problem. This value measures how well a heuristic solves a task such as Bi-TSP or Bi-CVRP. The *outer hypervolume* is computed over the population of generated heuristics, where each heuristic is represented by a score vector of the form $(-HV_{\text{inner}}, runtime)$. The first coordinate is negated so that the outer population manager can minimize both coordinates.

This distinction is essential when interpreting the experimental results. The figures

and tables in Chapter 5 report heuristic-population hypervolume and AUBC under the experimental framework used in this thesis. They should not be confused with the task-level solution hypervolume tables reported in the original MPaGE paper.

The two hypervolume levels have different reference points because they live in different spaces. For an inner Bi-TSP archive, the points are tour-cost vectors. For an inner Tri-TSP archive, the points are three-dimensional tour-cost vectors. For the outer population, however, every heuristic is represented by a two-dimensional score vector $(-HV_{\text{inner}}, runtime)$, even when the inner task is tri-objective. Thus, the outer hypervolume reference point is a bound in heuristic-score space, not in solution-objective space. This distinction is important because confusing the two spaces can lead to incorrect AUBC computation and misleading comparisons.

The outer hypervolume is also sensitive to the range of the first score component. For tasks where inner hypervolume values are large, such as Tri-TSP, the dominated extent along the negated-inner-HV axis can be much larger than in Bi-TSP. Consequently, absolute outer-HV values should not be compared directly across tasks. The thesis therefore uses within-cell normalized scores and ranks when summarizing performance across task families. Absolute values remain meaningful within a task–budget cell, where all methods are evaluated under the same task, budget, reference point, and aggregation convention.

In the implementation, hypervolume computation is centralized in the reward module. When `pymoo` is available, the implementation uses `pymoo.indicators.hv.HV` [7]; otherwise, it falls back to an internal two-dimensional implementation for supported cases.

2.6 MPaGE

MPaGE, Pareto-Grid-Guided Large Language Models, is the direct methodological baseline for this thesis [1]. It frames heuristic design as an LLM-guided program search problem for multi-objective combinatorial optimization. Its workflow combines generated Python heuristics, execution-based evaluation, population management, Pareto Front Grid selection, and semantic clustering.

The first key component is a SEMO-style heuristic population. Each generated heuristic is evaluated and inserted into a population according to multi-objective criteria. The population favors non-dominated heuristics and uses diversity mechanisms to avoid collapsing to a narrow set of similar programs. This population-management design is inherited from the original MPaGE framework.

The second key component is Pareto Front Grid (PFG) selection. PFG discretizes the heuristic objective space and selects representatives from different grid regions. This mechanism is designed to preserve useful variation along the heuristic Pareto front. The parameters

`pfg_gk`, `pfg_sigma`, and `pfg_epsilon` are retained in the BMAB implementation.

The third key component is LLM-based semantic clustering. Rather than grouping heuristics only by numerical score, MPaGE asks an LLM to cluster heuristic descriptions or code by algorithmic similarity. Mutation is then applied within a semantic cluster, while crossover can combine heuristics across clusters. The implementation retains the original MPaGE prompt families for initialization, mutation, crossover, and clustering.

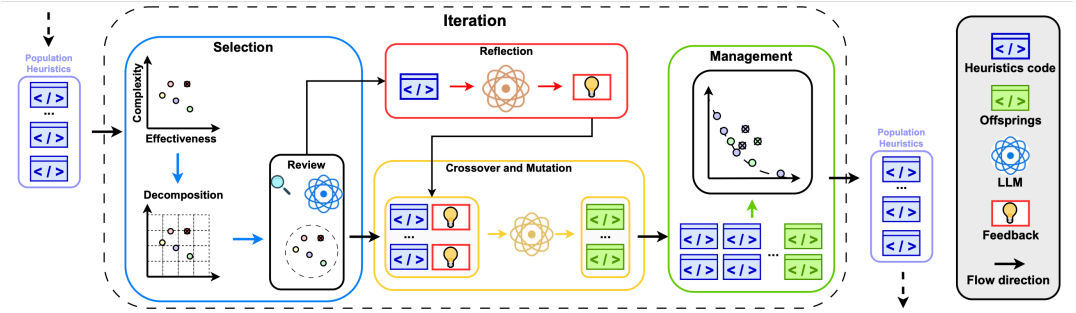


Figure 2.1: Overview of the MPaGE framework, adapted from the original MPaGE materials.

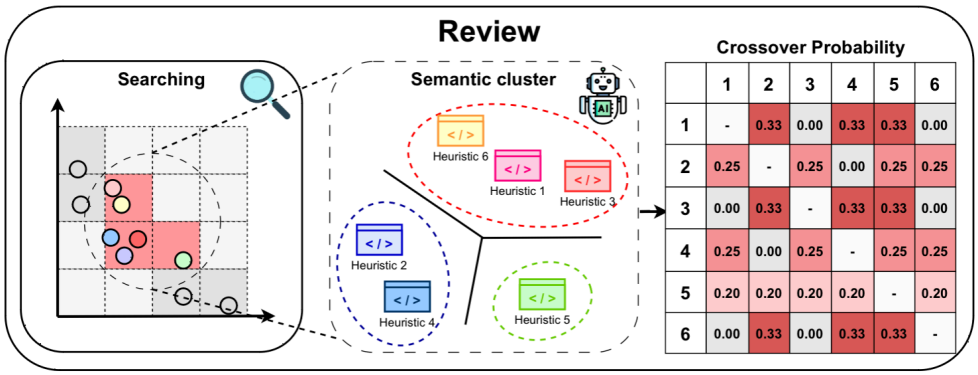


Figure 2.2: Pareto Front Grid selection in MPaGE, adapted from the original MPaGE materials.

The proposed BMAB-LLM method does not replace these components. Instead, it keeps the MPaGE generation and evaluation substrate and modifies the scheduling policy that decides how to spend LLM calls.

2.7 LLM-Based Program Generation for Heuristic Design

The proposed framework belongs to the broader area of LLM-based program synthesis with execution feedback. In systems such as FunSearch, an LLM generates candidate programs, an evaluator scores them, and the best programs are retained for further search [8]. MPaGE

and BMAB-LLM apply the same general principle to heuristic design: generated code is not accepted because it is plausible natural language output, but because it passes execution and improves a measurable optimization criterion.

In the implementation used in this thesis, the LLM is used primarily for prompt-driven code generation and semantic clustering. The prompt templates request Python functions with a specified signature. The secure evaluation layer then executes the generated function under task-specific constraints. This design creates a closed loop: generation proposes code, evaluation supplies a score, and population management determines whether the generated heuristic should influence future prompts.

The source artifacts do not contain evidence of LLM fine-tuning, supervised training, LoRA, PEFT, knowledge distillation, retrieval-augmented generation, diffusion models, or GAN-based generation. The generative component is therefore best described as prompt-based program generation using an external LLM API.

Execution feedback is the key mechanism that makes prompt-based generation usable for optimization. The LLM is not assumed to know whether a generated heuristic is correct or effective. Instead, correctness and usefulness are determined by running the generated code. This makes the evaluator a central part of the learning loop: it converts an untrusted program string into either a numerical score or a failure signal. The approach is similar in spirit to program-search systems such as FunSearch [8], where the model proposes code and the environment supplies objective feedback.

The downside is that execution feedback is expensive and noisy. A generated function may look plausible in natural language but fail at runtime because it uses the wrong variable name, returns an infeasible object, or violates a hidden representation constraint. Even valid code may be behaviorally redundant, producing a neighbor operator that is nearly identical to an existing heuristic. The proposed BMAB controller is designed for precisely this setting: rewards are sparse, failures are common enough to matter, and exploration must be controlled under a finite call budget.

2.8 Execution Failures, Null Scores, and Program-Search Failure Modes

In the implementation used in this thesis, an invalid heuristic is not merely a low-quality heuristic. It may fail before producing any meaningful score. The evaluation infrastructure can return a null score when parsing, execution, timeout handling, or task-specific validation fails. These null outcomes are important because they consume budget without directly improving the heuristic population.

There are several qualitatively different failure modes. A syntactic failure means the

LLM output cannot be parsed as executable Python. An interface failure means the function exists but does not match the evaluator's expected signature or return type. A semantic failure means the function runs but returns an infeasible neighbor, such as an invalid tour, overloaded route, or capacity-violating knapsack vector. A runtime failure means the function exceeds the allowed evaluation time or raises an exception during repeated inner-loop calls. The current thesis focuses on AUBC and final outer hypervolume as the main performance evidence; finer-grained failure logging would be valuable future work.

Executability matters differently from final quality. A method that produces many executable heuristics has more opportunities to improve the outer population, but executability alone does not guarantee high hypervolume. An executable heuristic can still be dominated, redundant, or too slow. The central empirical question is therefore whether budget-aware scheduling produces programs that contribute useful objective-space trade-offs under a fixed call budget.

2.9 Adaptive Operator Selection

Adaptive operator selection studies how an evolutionary algorithm can choose among variation operators based on their observed usefulness during the run. Early work shows that reward-based operator control can improve search by allocating more trials to operators that produce valuable offspring [9]. This idea is directly relevant to LLM-based heuristic design because mutation and crossover prompts do not have uniform value across all stages of the search.

The difference is the cost model. In a conventional evolutionary algorithm, applying a mutation operator may be computationally cheap. In this thesis, applying a mutation or crossover prompt consumes an LLM call. As a result, operator selection is not merely a performance-tuning mechanism; it is a budget-allocation policy. This observation motivates the operator-level UCB component of the proposed scheduler.

A second difference is semantic locality. A mutation prompt is usually expected to exploit a single parent by changing or improving its algorithmic idea. A crossover prompt combines two parents and may explore a larger region of program space. Neither operator is uniformly preferable. Mutation can be effective when a cluster already contains strong heuristics whose local variants are likely to remain valid. Crossover can be useful when the current population contains complementary ideas in different semantic clusters. The operator controller therefore has to learn from the observed reward stream rather than relying on a fixed schedule.

2.10 Multi-Armed Bandits and UCB

A multi-armed bandit models sequential decision-making under uncertainty. At each step, the algorithm selects an arm and observes a reward. It must balance exploitation of arms with high empirical reward and exploration of arms whose reward estimates remain uncertain. UCB1 formalizes this trade-off with an optimism bonus [10, 11]:

$$\mathrm{UCB}(a) = \hat{\mu}_a + c\sqrt{\frac{\log t}{n_a}}, \tag{2.6}$$

where $\hat{\mu}_a$ is the empirical mean reward of arm a, n_a is the number of times arm a has been selected, t is the total number of selections, and c controls exploration.

BMAB-LLM uses this principle at two levels. The operator bandit persists across generations and chooses between mutation and crossover. The cluster bandit is reconstructed within a generation and chooses semantic cluster–operator arms. This two-level structure reflects the fact that operator identities are stable across the entire run, whereas cluster identities are local to a particular clustering call.

The bandit formulation is an approximation, not a claim that the heuristic-design environment is a perfectly stationary stochastic bandit. Rewards depend on the current population, the LLM response, the evaluator, and the remaining budget. Nevertheless, the bandit abstraction is useful because it provides an explicit mechanism for balancing empirical success and uncertainty. It also gives a disciplined alternative to ad hoc scheduling rules such as always alternating mutation and crossover or selecting clusters uniformly.

In the thesis setting, an arm pull corresponds to spending at least one LLM call. The reward is computed after the generated child is evaluated. This delayed feedback is still short enough to be used online: after every candidate heuristic, the operator and cluster statistics can be updated before choosing the next candidate. Thus, the method creates a closed loop between generation, evaluation, and future call allocation.

2.11 Budgeted Bandits

Budgeted bandits extend the standard bandit setting by assigning costs to actions and requiring the policy to operate under a finite resource constraint [12]. This is a natural model for LLM-based search, where each call to the generator or clusterer consumes budget. A method that ignores cost may obtain high reward only by using many calls inefficiently, which is undesirable in practical settings.

The implementation exposes this constraint through a hard `BudgetTracker`. The bandit scheduler uses reward-per-cost at the operator level and a budget-aware exploration term at the cluster level. This is not claimed to be an optimal solution to all budgeted bandit

formulations. Rather, it is a pragmatic scheduling mechanism that is compatible with the MPaGE loop and with the thesis objective of improving hypervolume progress per LLM call.

Formally, if action a_t at decision time t produces reward r_t and consumes cost c_t, the budgeted objective can be written as

$$\max_{\pi} \mathbb{E}_{\pi}\left[\sum_{t=1}^{T} r_t\right] \quad \text{subject to} \quad \sum_{t=1}^{T} c_t \leq B, \tag{2.7}$$

where π is the scheduling policy and B is the available call budget. In the current implementation, the most important costs are clustering calls and heuristic-generation calls. Both are charged explicitly. The hard budget tracker ensures that the method cannot silently exceed the configured budget, which is essential for fair comparison with MPaGE-orig.

The remaining-budget term in the cluster bandit has a practical interpretation. Early in a run, exploration is valuable because the controller has little evidence about which semantic regions are useful. Near the end of a run, exploration becomes riskier because a failed exploratory call cannot be compensated by many later calls. Scaling the exploration bonus by the remaining budget fraction therefore shifts the policy from exploratory to exploitative behavior as the horizon approaches. This does not make the policy optimal, but it matches the operational goal of protecting final hypervolume while still allowing early discovery.

2.12 Page–Hinkley Drift Detection

The reward distribution in heuristic design is non-stationary. A semantic cluster may initially contain promising parents, but after repeated exploitation its marginal value can decrease. Conversely, a previously weak cluster may become useful after the population changes. Standard UCB estimates can become overconfident when the reward distribution changes.

Page–Hinkley is an online change-detection test for shifts in the mean of a sequential signal [13]. In the proposed scheduler, `PageHinkleyState` tracks a running mean and a cumulative deviation statistic. When the gap between the historical maximum and the current cumulative statistic exceeds a threshold, the corresponding cluster arm is treated as stale and its statistics are reset. The default parameters used in the experiments are $\delta = 0.005$ and threshold 0.5.

2.13 AUBC: Area Under the Budget Curve

For a budget-constrained search method, the final value alone does not capture how efficiently the method uses its budget. AUBC addresses this issue by integrating hypervolume

over consumed budget:

$$\text{AUBC} = \frac{1}{B} \int_0^B HV(b)\, db, \tag{2.8}$$

where $HV(b)$ is the outer heuristic-population hypervolume after consuming budget b. The profiler records budget-HV points, anchors the curve at $(0, 0)$, and extends the last observed value to the total budget when necessary. This is the metric formulation used for all AUBC values reported in Chapter 5.

AUBC is particularly appropriate for this thesis because the goal is not only to find a strong final heuristic population, but also to spend LLM calls productively throughout the run. A method with higher AUBC can be preferable even when its final hypervolume is similar, especially in interactive or cost-sensitive settings where early stopping is possible.

The normalization by B makes AUBC interpretable as an average hypervolume value over the budget horizon. If two methods end with the same $HV(B)$, the method with the higher AUBC reached useful populations earlier. If one method has higher final HV but lower AUBC, the result indicates a trade-off between early progress and terminal quality. Chapter 5 uses exactly this distinction: final HV identifies the quality of the final managed population, while AUBC identifies how much value was obtained over the entire budget trajectory.

The profiler anchors the budget curve at $(0, 0)$ before the first recorded point. This convention reflects the fact that before any budget is consumed, no generated heuristic population exists under the run. The profiler also extends the last observed hypervolume to the total budget when the final recorded point does not exactly equal B. This prevents underestimating methods whose last population update occurs slightly before the budget is exhausted.

2.14 Technique Inventory

Table 2.1 summarizes the generative AI and machine learning techniques that are either present or absent in the implementation. The table is included to keep the scope of the thesis precise: only techniques with direct evidence in the codebase or documentation are analyzed as part of this study.

Table 2.1: Inventory of generative AI and machine learning techniques in this thesis.

Technique	Status	Evidence in this thesis
Large language models	Present	The generator and semantic clusterer are external LLM calls controlled by the BMAB workflow.

Technique	Status	Evidence in this thesis
Prompt engineering	Present	The implementation uses initialization, mutation, crossover, and semantic-clustering prompt families inherited from MPaGE.
Bandits / lightweight reinforcement learning	Present	The scheduler implements UCB-style operator selection, budget-aware cluster selection, and Page–Hinkley drift handling.
Execution-based evaluation of LLM-generated code	Present	Generated Python heuristics are executed by task evaluators, and failures can produce null scores.
RAG	Not found	No information was found in the source code or documentation.
Fine-tuning, PEFT, LoRA	Not found	No information was found in the source code or documentation.
Diffusion models, GANs	Not found	No information was found in the source code or documentation.
Knowledge distillation	Not found	No information was found in the source code or documentation.
Multi-agent system	Not found	The system contains an LLM–evolution loop and two bandit controllers, but no evidence was found for multiple autonomous agents.

2.15 Limitations of Existing Approaches

The baseline MPaGE framework provides a strong foundation for LLM-based multi-objective heuristic design, but it does not explicitly optimize the allocation of LLM calls under a hard budget. Its semantic clustering and PFG selection maintain diversity, yet they do not by themselves answer which operator and which semantic region should receive the next call. This leaves room for waste when some clusters produce invalid code, redundant heuristics, or low marginal hypervolume improvement.

BMAB-LLM addresses this gap by treating generation as a sequential budgeted decision problem. The method is still affected by the stochasticity of external LLM APIs, the possibility of invalid generated code, and evaluator runtime costs. These factors motivate both the implementation safeguards discussed in Chapter 4 and the careful empirical interpretation applied in Chapter 5.

Chapter 3

Proposed Method

3.1 Research Motivation and Design Rationale

The proposed method, denoted BMAB-LLM, is designed as a budget-aware extension of the original MPaGE framework. The central observation behind this thesis is that LLM-driven heuristic design is not only an optimization problem over generated programs, but also a resource-allocation problem. Each heuristic-generation, clustering, or optional reflection step consumes an LLM call, and therefore consumes time, money, and limited experimental budget. A method that produces high-quality heuristics only after many unproductive calls is less useful than a method that converts early calls into a strong heuristic Pareto front.

The original MPaGE paper formulates automatic heuristic design for multi-objective combinatorial optimization problems (MOCOPs) as a Language Multi-Criteria Heuristic Design problem. It combines three important ingredients: a SEMO-style heuristic population, Pareto-Front-Grid (PFG) elite selection, and LLM-based semantic clustering of heuristic code [1, 4]. This design is well motivated because it searches for a population of heuristics that trade off solution quality and runtime, rather than a single best heuristic. It also recognizes that code-level diversity is not equivalent to semantic diversity: two heuristics may be syntactically different but implement nearly identical search logic.

However, the baseline MPaGE workflow allocates generation effort in a mostly predetermined manner. Each generation repeatedly selects elites with PFG, groups promising heuristics into semantic clusters, applies mutation or crossover prompts, evaluates the generated program, and updates the population. This loop is effective for maintaining semantic and objective-space diversity, but it does not explicitly learn which operator or which semantic region is producing the most useful heuristics under the current budget. The stopping condition is also naturally expressed in terms of iterations or sample counts, whereas practical LLM-based search is constrained by a finite number of API calls. Under a strict budget B, the system should therefore learn where to spend the remaining LLM calls instead

of treating all operators and clusters as equally promising.

BMAB-LLM therefore keeps the productive heuristic-generation mechanisms of MPaGE and replaces only the scheduling layer. In particular, the implementation preserves the original task templates, prompt taxonomy, secure evaluator, PFG-based population management, and LLM semantic clustering implementation. The new contribution is a two-level budgeted multi-armed bandit controller that decides which variation operator and which semantic cluster should receive each LLM call. This design follows a conservative research principle: the comparison against MPaGE is made meaningful by reusing the same generation and evaluation substrate, while changing the decision policy that allocates budget.

3.2 Problem Formulation

3.2.1 Inner Multi-Objective Combinatorial Optimization Problem

Let $\mathcal{X}$ be a finite or discrete feasible set for a MOCOP, and let

$$f(x) = (f_1(x), f_2(x), \ldots, f_M(x)), \qquad x \in \mathcal{X}, \tag{3.1}$$

where all objectives are minimized. For two feasible solutions $x_a, x_b \in \mathcal{X}$, x_a Pareto-dominates x_b, written $x_a \prec x_b$, if

$$f_i(x_a) \leq f_i(x_b) \quad \forall i \in \{1, \ldots, M\}, \qquad \exists j : f_j(x_a) < f_j(x_b). \tag{3.2}$$

The Pareto front is the image of the non-dominated solution set in objective space. Its quality is commonly summarized by the hypervolume indicator $HV_r(\cdot)$ with respect to a reference point r [5, 6]. For a finite objective set Y in a minimization problem, the hypervolume is the Lebesgue measure of the region dominated by the non-dominated points in Y and bounded by r:

$$HV_r(Y) = \lambda\left(\bigcup_{y \in \mathrm{ND}(Y)} [y_1, r_1] \times \cdots \times [y_M, r_M]\right),$$

where $\mathrm{ND}(Y)$ denotes the non-dominated subset and $\lambda(\cdot)$ denotes M-dimensional volume. A larger hypervolume means that the obtained Pareto approximation is both closer to the ideal region and more broadly spread across trade-offs. The benchmark evaluators in this thesis use this quantity as the inner solution-quality signal and execution time as the efficiency signal.

3.2.2 Outer Heuristic-Design Problem

Following the LMHD formulation in the original MPaGE paper, a heuristic $h \in \mathcal{H}$ is treated as a program that maps the current SEMO archive to a new candidate solution:

$$h : 2^{\mathcal{X}} \to \mathcal{X}, \qquad x' = h(A), \tag{3.3}$$

where A is the current archive of non-dominated solutions maintained by the inner solver. The evaluator assigns each generated heuristic a vector-valued score

$$E(h) = (e_1(h), e_2(h)) = (-HV_{\text{inner}}(h), T(h)) \,. \tag{3.4}$$

Equation 3.4 should be read as the interface between heuristic design and the inner solver. The LLM does not directly output a TSP tour, CVRP route, or knapsack solution. Instead, it outputs a heuristic program h. During evaluation, the inner SEMO-style solver repeatedly calls h with its current archive A and expects one candidate solution x'. The quality of h is therefore determined indirectly by the Pareto archive that the inner solver obtains after running with that heuristic.

Here $HV_{\text{inner}}(h)$ is the hypervolume achieved by the solutions produced when h is used inside the benchmark solver, and $T(h)$ is the measured runtime. More concretely, if the inner run using h returns a non-dominated solution archive A_h, then $HV_{\text{inner}}(h)$ is computed from the objective vectors $\{f(x) : x \in A_h\}$ using the task-specific reference point. This is the *inner hypervolume*: it measures how good one generated heuristic is at solving the underlying combinatorial optimization task.

The first coordinate in $E(h)$ is negated because the outer population-management code is written as a minimization procedure. Thus, a heuristic with a larger inner hypervolume receives a smaller first score coordinate. The second coordinate is runtime, so smaller is also better. This score convention is implemented by the benchmark evaluators in the vendored MPaGE task modules.

At the outer level, the algorithm maintains a population $\mathcal{P}_t \subset \mathcal{H}$ of valid heuristics and aims to approximate the non-dominated set in the heuristic score space. Let $F_t = \{E(h) : h \in \mathcal{P}_t\}$ be the outer heuristic front at step t. Given an LLM-call budget B, BMAB-LLM is optimized for two related criteria:

$$\max \; HV_{\text{outer}}(B) = \max \; HV_r(F_B), \tag{3.5}$$

$$\max \; \text{AUBC} = \max \; \frac{1}{B} \int_0^B HV_{\text{outer}}(b) \, db, \tag{3.6}$$

subject to

$$\sum_{t=1}^{T} c_t \leq B. \tag{3.7}$$

The quantity HV_{outer} is the *outer hypervolume*: it is computed over heuristic score vectors rather than over task solutions. A high outer hypervolume means that the final population contains useful trade-offs between inner solution quality and runtime, for example heuristics that achieve high inner HV quickly and heuristics that are slower but stronger. The final HV objective measures terminal population quality, while AUBC measures how rapidly the method converts budget into Pareto-front quality over the whole run. In the current experiments, the default cost model is call-based, i.e. one LLM request consumes one budget unit. The budget tracker also supports a token-labeled accounting mode, but the implemented main loop and reported experiments use fixed call costs.

3.3 System Overview

BMAB-LLM is organized as an adaptive orchestration layer around the MPaGE primitives. Figure 3.1 summarizes the data and control flow.

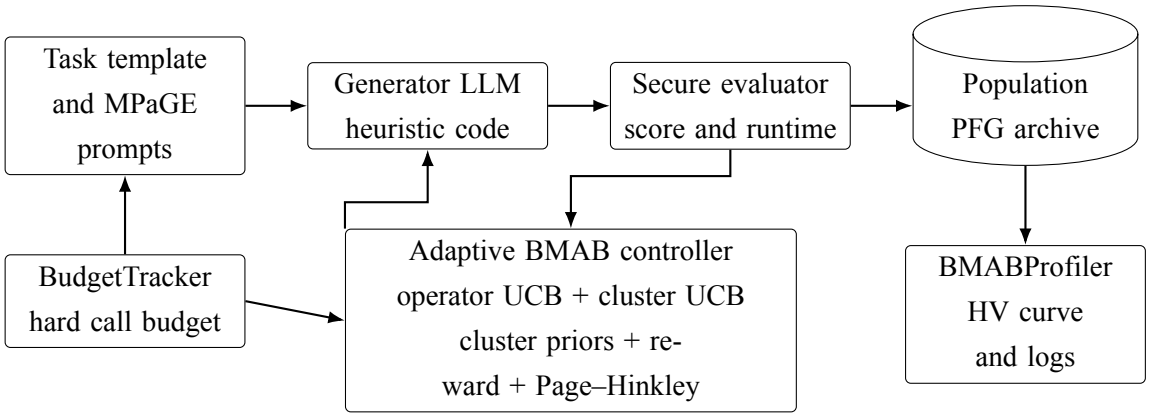


Figure 3.1: Logical architecture of BMAB-LLM. The method preserves MPaGE's prompt, evaluation, semantic clustering, and population-management components, while adding budget accounting and adaptive bandit control.

The implementation is centered in `bmab_llm.py`. The main supporting modules are summarized in Table 3.1.

Table 3.1: Main methodological modules in the implementation.

Module	Methodological role
`bmab_llm.py`	Main orchestrator. It owns the evolutionary loop, calls PFG selection, requests semantic clusters, selects bandit arms, samples offspring, computes rewards, and flushes the final population.
`budget.py`	Hard budget tracker. It stores total, remaining, and consumed budget and records a history entry for every charged LLM call.
`bandit.py`	Two-level adaptive scheduler: persistent operator UCB, per-generation cluster UCB, and Page–Hinkley drift detection.
`cluster_manager.py`	Wrapper around MPaGE semantic clustering. It charges cluster calls, validates or repairs partitions, and computes cluster-quality priors.
`reward.py`	Scalar credit assignment for bandit learning, including HVI, final managed-population HV gain, diversity gain, rank smoothing, and invalid-code penalties.
`profiler.py`	Records budget-vs-HV curves, AUBC, budget history, bandit state, and population artifacts for analysis.

3.4 Components Inherited from MPaGE

BMAB-LLM intentionally retains the core generative and evaluative substrate of MPaGE. This inheritance is documented in the implementation materials and is visible in the vendored MPaGE subtree. The retained components are important because they isolate the proposed contribution to budget-aware scheduling rather than changing the benchmark, prompt templates, or evaluation logic.

3.4.1 Population Representation and PFG Selection

Each individual is represented as a Python function together with a natural-language algorithm description, evaluation score, sample time, and execution time. The population class inherited from MPaGE maintains a managed set of heuristics using non-dominated sorting and diversity-preserving truncation.

PFG selection partitions the two-dimensional heuristic objective space into grid cells.

Let $P_t = \{E(h_i) : h_i \in \mathcal{P}_t\}$ be the current score set. For each objective $j \in \{1, 2\}$, define the ideal and nadir values

$$z_j^* = \min_{h \in \mathcal{P}_t} e_j(h), \qquad z_j^n = \max_{h \in \mathcal{P}_t} e_j(h). \tag{3.8}$$

With K_j segments and a small margin σ, the grid width is approximately

$$\Delta_j = \frac{z_j^n - z_j^* + \sigma}{K_j}. \tag{3.9}$$

A heuristic is assigned to a grid index

$$g_j(h) = \left\lfloor \frac{e_j(h) - z_j^*}{\Delta_j} \right\rfloor, \qquad G(h) = (g_1(h), g_2(h)). \tag{3.10}$$

Within each occupied cell, non-dominated representatives are retained. The union of retained representatives forms the elite set used for reproduction. This mechanism is inherited from MPaGE and is implemented in the vendored MPaGE population manager. In the current BMAB loop, the PFG-selected slice is used as a high-quality signal for parent weighting, while the wider managed population is made available to the clustering layer to avoid over-narrow clusters.

3.4.2 LLM-Based Semantic Clustering

The original MPaGE paper argues that objective-space or syntax-only diversity is insufficient for heuristic design. Two code snippets can have different syntax but the same algorithmic logic, and two heuristics with similar performance may use distinct search strategies. MPaGE addresses this by prompting an LLM to group elite heuristics according to semantic and behavioral similarity.

Formally, for a candidate set $P = \{h_1, \ldots, h_n\}$, the clustering module returns

$$\text{SemClust}(P) = \{C_1, \ldots, C_K\}, \tag{3.11}$$

such that

$$\bigcup_{k=1}^{K} C_k = P, \qquad C_i \cap C_j = \emptyset \ (i \neq j). \tag{3.12}$$

The implementation calls the MPaGE cluster prompt through `cluster_manager.py`. If the LLM response is malformed, incomplete, or unavailable, the method falls back to singleton clusters. This design preserves semantic clustering when possible while ensuring that the evolutionary loop remains robust to LLM-format failures.

3.4.3 Mutation and Crossover Prompt Families

MPaGE defines a prompt taxonomy for initialization, mutation, and crossover. BMAB-LLM keeps this taxonomy. Initialization uses the I1 prompt to create zero-shot heuristics. Mutation uses M1 or M2 to modify a single parent from the selected cluster. Crossover uses E1 or E2 to combine one parent from the selected cluster with a second parent outside that cluster. Thus, BMAB preserves MPaGE's core semantic rule:

$$\text{mutation} = \text{intra-cluster local refinement}, \qquad \text{crossover} = \text{inter-cluster recombination}. \tag{3.13}$$

The difference is not the prompt content but the policy that decides when a mutation or crossover prompt should be used, and which cluster should provide the parent.

3.5 Hard Budget Model

The LLM-call budget is treated as a first-class constraint. Let B be the total budget and b_t be the remaining budget before action t. Each LLM action has cost c_t, and the transition is

$$b_{t+1} = b_t - c_t, \qquad b_t \geq c_t. \tag{3.14}$$

If the system cannot afford an action, the action is not issued. The current implementation charges budget before the heuristic-generation call or cluster call is sent. This is important experimentally: invalid code, parsing failure, timeout, and non-improving offspring are still charged because the API resource has already been consumed.

In the call-based mode used by the reported experiments, the following costs are used:

Table 3.2: Budget costs used by the current implementation.

Action	Cost
Initialization call (I1)	1
Heuristic-generation call (M1/M2/E1/E2)	1
Semantic clustering call	1
Optional review/suggestion call	1

The budget state is recorded through `BudgetTracker.history`, and the profiler later writes `budget_history.json`. This gives the experimental harness a direct account of how the budget was spent.

3.6 Two-Level Budgeted Bandit Scheduler

3.6.1 Why a Two-Level Scheduler is Needed

A direct bandit over individual heuristics is not viable because the heuristic space is open-ended: every LLM call can create a previously unseen program. A direct persistent bandit over semantic cluster IDs is also problematic because clusters are reconstructed at each generation and their numeric labels are not stable across generations. BMAB-LLM therefore separates the adaptive decision into two levels:

1. a persistent operator-level bandit over the stable action set $\mathcal{O} = \{\mu, \chi\}$, where μ denotes mutation and χ denotes crossover;
2. a per-generation cluster-level bandit over arms (k, o), where k indexes one semantic cluster in the current generation and $o \in \mathcal{O}$ is the selected operator.

This decomposition lets the method learn long-term operator utility while still adapting to the short-lived semantic cluster structure of the current population.

3.6.2 Operator-Level UCB

For each operator $o \in \mathcal{O}$, the operator bandit stores the number of pulls n_o, cumulative reward S_o^R, and cumulative cost S_o^c. Its empirical utility is reward per cost:

$$\hat{\rho}_o = \frac{S_o^R}{\max(S_o^c, \varepsilon)}. \tag{3.15}$$

The selected operator maximizes a UCB score:

$$U_{\text{op}}(o) = \hat{\rho}_o + c_{\text{op}} \sqrt{\frac{2 \log N_{\text{op}}}{n_o}}, \tag{3.16}$$

where $N_{\text{op}} = \sum_{o' \in \mathcal{O}} n_{o'}$ and c_{op} controls exploration. The implementation initializes each operator with an optimistic prior count and prior reward, preventing division by zero and encouraging early exploration. This is implemented in `bandit.py` by the `OperatorBandit` class.

3.6.3 Cluster-Level Budgeted UCB

At the beginning of each generation, `ClusterManager` returns a partition $\{C_1, \ldots, C_K\}$ and a quality prior q_k for each cluster. At this point, q_k can be understood as a scalar estimate of how promising cluster C_k is before any new offspring are generated in the current

generation; its formal front-aware definition is given in Section 3.7. The cluster bandit then creates arms (k, o) for all clusters and operator labels. Each arm stores cumulative reward and cumulative cost, and its exploitation term is

$$\hat{\rho}_{k,o} = \frac{S^R_{k,o}}{\max(S^c_{k,o}, \varepsilon)}. \tag{3.17}$$

The cluster-level UCB score is

$$U_{\text{cl}}(k, o) = \hat{\rho}_{k,o} + c_{\text{cl}}\alpha_t \sqrt{\frac{2 \log N_{\text{cl}}}{n_{k,o}}}, \tag{3.18}$$

where

$$\alpha_t = \left(\frac{b_t}{B}\right)^{\gamma_b} \tag{3.19}$$

when budget annealing is enabled, and $\alpha_t = 1$ otherwise. The factor α_t decreases as the budget is exhausted, making the controller more exploitative near the end of the run. This directly addresses the finite-horizon nature of the task: late calls have little time to recover from risky exploration.

The cluster bandit is warm-started from both cluster quality and operator priors. Let p_o be the softmax-normalized operator utility from the persistent operator bandit. The initial reward assigned to arm (k, o) is proportional to

$$r^{(0)}_{k,o} = \frac{1}{2}\left(q_k + p_o\right). \tag{3.20}$$

This warm start transfers information from the persistent operator layer into the generation-local cluster layer without assuming that cluster IDs persist across generations.

3.7 Front-Aware Cluster Priors

The semantic cluster partition identifies behavioral groups, but it does not by itself rank those groups. The current implementation therefore computes a front-aware prior for each cluster. For cluster C_k, define $S_k = \{E(h) : h \in C_k\}$ and $S = \{E(h) : h \in \mathcal{P}_t\}$. With outer reference point r, the hypervolume contribution of the cluster is

$$\Delta HV_k = \max\left(0,\ HV_r(S) - HV_r(S \setminus S_k)\right). \tag{3.21}$$

The heuristic scores in this section are the two-dimensional evaluator outputs $s = E(h) = (-HV_{\text{inner}}(h), T(h))$. Therefore, s_1 represents negated inner hypervolume and s_2

represents runtime. Because the first score coordinate is negative inner HV, a simple quality proxy for solution strength is

$$I_k = \max\left(0,\ -\min_{s \in S_k} s_1\right), \tag{3.22}$$

with a finite cap in the implementation to avoid an excessively large prior. A runtime proxy is defined as

$$T_k = \frac{1}{1 + \min_{s \in S_k} s_2}. \tag{3.23}$$

The unnormalized prior is

$$\tilde{q}_k = \Delta HV_k + 0.25 I_k + T_k, \tag{3.24}$$

and the normalized prior used by the bandit is

$$q_k = \frac{\tilde{q}_k}{\max_j \tilde{q}_j} \tag{3.25}$$

when the denominator is positive. This design is implemented in `cluster_manager.py`. It is more aligned with the final outer-front objective than a one-dimensional score proxy because it rewards clusters that currently contribute to the heuristic Pareto front while still accounting for inner solution quality and runtime.

3.8 Reward and Credit Assignment

3.8.1 Reward Components

The reward function converts the outcome of one LLM generation call into a scalar signal for the two bandits. For a valid generated heuristic with score s, the reward is

$$R(s) = w_q Q(s) + w_d \max(0, \Delta D) + w_r \text{Rank}(s). \tag{3.26}$$

If the generated heuristic is invalid, cannot be parsed, fails evaluation, or returns a non-finite score, the reward is

$$R(s) = -\lambda_{\text{pen}}. \tag{3.27}$$

The values used in the current implementation are $w_q = 1.0$, $w_d = 0.3$, $w_r = 0.2$, and $\lambda_{\text{pen}} = 1.0$. These settings are exposed through the command-line interface for the weights and fixed in `RewardComputer` for the penalty.

3.8.2 Quality Signal

Let F be the set of valid population scores before the new candidate is inserted. The immediate hypervolume improvement is

$$\mathrm{HVI}(s; F) = \max\left(0,\ HV_r(F \cup \{s\}) - HV_r(F)\right). \tag{3.28}$$

The implementation also computes a managed-population HV delta. Let $\mathrm{Manage}_N(\cdot)$ denote the same score-only survivor-selection logic used by the managed population with cap N. Then

$$\Delta HV_{\text{managed}} = \max\left(0,\ HV_r(\mathrm{Manage}_N(F \cup \{s\})) - HV_r(\mathrm{Manage}_N(F))\right). \tag{3.29}$$

Both quantities are normalized by rolling maxima over a recent window. Let $\widehat{\mathrm{HVI}}$ and $\widehat{\Delta HV}_{\text{managed}}$ denote the normalized values. The quality mode is then

$$Q(s) = \begin{cases} \widehat{\mathrm{HVI}}(s; F), & \text{mode} = \text{dense}, \\ \frac{1}{2}\widehat{\mathrm{HVI}}(s; F) + \frac{1}{2}\widehat{\Delta HV}_{\text{managed}}(s; F), & \text{mode} = \text{hybrid}, \\ \widehat{\Delta HV}_{\text{managed}}(s; F), & \text{mode} = \text{final\_hv}. \end{cases} \tag{3.30}$$

The experimental framework evaluates three parallel quality-signal variants while retaining the same BMAB scheduling pipeline. Final-HV reward uses `final_hv`, Dense reward uses `dense`, and Hybrid reward uses `hybrid`. This design makes it possible to compare whether terminal managed-population HV, immediate HVI, or a mixture of the two provides the most useful credit signal.

3.8.3 Diversity and Rank Smoothing

Pure HVI is often sparse: many valid offspring do not immediately expand the outer Pareto front. To provide a denser signal, BMAB-LLM adds rank and diversity terms. The cumulative diversity index used in the reward is the mean pairwise Euclidean distance among valid score vectors:

$$D(F) = \frac{1}{|F|(|F|-1)} \sum_{i \neq j} \|s_i - s_j\|_2. \tag{3.31}$$

The diversity increment is $\Delta D = D(F \cup \{s\}) - D(F)$. Only positive increments are rewarded. The rank score is

$$\mathrm{Rank}(s) = \frac{1}{|F|}\left|\{u \in F : \exists j,\ s_j < u_j\}\right|. \tag{3.32}$$

In this equation, s and u are heuristic scores of the form $E(h) = (-HV_{\text{inner}}(h), T(h))$. The first coordinate measures negated inner hypervolume, and the second coordinate measures runtime. Since both coordinates are minimized in the outer heuristic population, a smaller coordinate value indicates either higher inner solution quality or lower execution time. The rank term therefore rewards a candidate that improves at least one objective relative to existing population members, even when it does not immediately increase HV.

3.9 Non-Stationarity and Page–Hinkley Reset

The reward distribution in evolutionary heuristic design is non-stationary. A cluster that is useful early may become unproductive after its design pattern has already been exploited. To reduce overconfidence in stale arm statistics, each cluster-level arm is monitored by a Page–Hinkley detector [13].

For a reward sequence (r_t) of one arm, the implementation maintains the running mean $\bar{r}_t$ and cumulative statistic

$$\bar{r}_t = \bar{r}_{t-1} + \frac{r_t - \bar{r}_{t-1}}{t}, \tag{3.33}$$

$$m_t = m_{t-1} + r_t - \bar{r}_t + \delta, \tag{3.34}$$

$$M_t = \max(M_{t-1}, m_t). \tag{3.35}$$

Downward drift is declared when

$$M_t - m_t > \lambda_{\text{PH}}. \tag{3.36}$$

When drift is detected, the corresponding arm statistics are reset to an optimistic prior for the current generation. This mechanism is implemented in `bandit.py`. It is limited to the cluster-level bandit because cluster rewards are local to the current generation and most sensitive to rapid changes in population state.

3.10 Parent Selection and Variation Workflow

The original MPaGE strategy uses intra-cluster mutation and inter-cluster crossover. BMAB-LLM preserves this logic but changes how parents are sampled. The loop first obtains PFG-selected elites through `Population.selection`. It then expands the clustering pool to the entire current managed population to give the cluster bandit a richer set of semantic groups. To prevent the wider pool from diluting the PFG signal, parent sampling gives candidates that also appear in the PFG-selected elite set a larger weight:

$$w(h) = \begin{cases} 3, & h \in E_{\text{PFG}}, \\ 1, & \text{otherwise.} \end{cases} \tag{3.37}$$

For mutation, a single parent is sampled from the selected cluster using these weights. For crossover, the first parent is sampled from the selected cluster and the second parent is sampled from other clusters when possible. This preserves semantic recombination while biasing the process toward strong PFG representatives. In the implementation, this logic is used when constructing the mutation or crossover prompt and when sampling parents from the selected semantic groups.

3.11 Complete Algorithm

Algorithm 3.1 gives the high-level workflow implemented by `BMABLLM.run`. The pseudocode abstracts away exception handling and API-specific details, but preserves the execution order that matters methodologically: initialization, budget-aware clustering, adaptive operator and cluster selection, reward computation, bandit update, drift handling, and final population flushing.

Listing 3.1: High-level BMAB-LLM workflow.

```
Algorithm BMAB-LLM
Input: evaluator E, generator L, clusterer C,
     budget B, population cap N, reference point r
Output: managed heuristic population P

initialize BudgetTracker(B), Population P
initialize OperatorBandit, ClusterBandit, RewardComputer

while initialization target is not reached
     and one generation call is affordable do
   charge one budget unit
   h <- L(I1 initialization prompt)
   score <- E(h)
   if score is valid then
      register h in P
   end if
end while

record the initial budget-HV point
```

```
while budget remains do
   E_pfg <- PFG-selected elites from P
   H_all <- current managed population

   if a clustering call and a generation call are affordable then
      charge one budget unit
      clusters <- C(H_all)
      if clusters are invalid then
         clusters <- singleton fallback clusters
      end if
   else
      clusters <- singleton fallback clusters
   end if

   q <- front-aware cluster priors
   p <- operator priors from OperatorBandit
   reset ClusterBandit using clusters, q, and p

   produced <- 0
   while produced < N
        and one generation call is affordable do
      op <- select mutation or crossover
      arm <- select a cluster arm for op
      parents <- weighted parent sampling from clusters
      prompt <- MPaGE mutation or crossover prompt

      F_before <- scores in P and pending valid offspring
      D_before <- diversity(F_before)

      charge one budget unit
      h_child <- L(prompt)
      score <- E(h_child)
      if score is valid then
         register h_child as pending
      end if

      F_after <- scores in P and pending valid offspring
      D_after <- diversity(F_after)
      reward <- RewardComputer(score, F_before,
                          D_before, D_after)
```

```
      update OperatorBandit(op, reward)
      update ClusterBandit(arm, reward)
      apply Page-Hinkley reset if drift is detected
      produced <- produced + 1
   end while

   merge pending valid offspring into P when needed
   record bandit state and budget-HV point
end while

flush all pending valid offspring into P
write budget history, budget curve, bandit log, and AUBC
```

3.12 Baseline and Variant Design

The MPaGE-orig baseline is not a new algorithm. It is a budget-instrumented wrapper around the original MPaGE driver that records budget consumption and budget-HV curves using the same profiler infrastructure as BMAB-LLM. The purpose is to compare allocation policies while keeping the task evaluators and generated artifacts as consistent as possible.

The current CLI defines the following method variants in `main.py`. The first three BMAB-style variants share the same scheduler and differ only in the quality signal $Q(s)$ used inside the reward:

Table 3.3: Main method variants relevant to the proposed framework.

Variant	Interpretation
Final-HV reward	BMAB pipeline with the `final_hv` quality signal, operator bandit, cluster bandit, Page–Hinkley, budget annealing, and PFG-weighted parent sampling.
Dense reward	BMAB pipeline with the immediate-HVI quality signal.
Hybrid reward	BMAB pipeline with a half immediate-HVI and half managed-final-HV quality signal.
`no_budget_anneal`	BMAB pipeline without remaining-budget scaling in the cluster UCB exploration term.
`no_ph`	Disables practical Page–Hinkley resets by using an extremely large threshold.
`no_diversity`	Removes the CDI diversity term by setting its weight to zero.
`op_only`	Disables the cluster bandit and samples clusters uniformly.
`cluster_only`	Disables the operator bandit and uses a round-robin operator sequence.
MPaGE-orig	Budget-instrumented wrapper around the original MPaGE implementation.

These variants are important because they separate the contribution of each methodological component. In particular, Final-HV reward, Dense reward, and Hybrid reward vary only the reward quality mode, while `no_budget_anneal`, `no_ph`, `no_diversity`, `op_only`, and `cluster_only` isolate specific control mechanisms.

Chapter 4

System Implementation

4.1 Implementation Organization

The implementation used in this thesis contains two closely related codebases. The first is the original MPaGE implementation, including its entry points, task evaluators, and documentation. The second is the extended implementation developed for this study, which provides the budget-aware BMAB controller, experiment harness, analysis artifacts, result files, and thesis source.

The main implementation modules of the extended framework are the following:

- `bmab_llm.py`: the main BMAB orchestration loop. It performs initialization, PFG selection, semantic clustering, bandit-based operator and cluster selection, offspring generation, reward computation, population update, and final artifact writing.
- `bandit.py`: the implementation of the operator-level UCB controller, cluster-level budgeted UCB controller, and Page–Hinkley drift detector.
- `budget.py`: the hard LLM-call budget tracker. It records consumed and remaining budget and rejects unaffordable actions.
- `cluster_manager.py`: the wrapper around LLM-based semantic clustering. It charges cluster calls, validates cluster partitions, computes cluster priors, and provides fallback behavior.
- `reward.py`: the reward computation module, including hypervolume, immediate HVI, managed final-HV gain, diversity gain, rank smoothing, and invalid-code penalties.
- `profiler.py`: the logging and measurement module that records budget curves, AUBC, bandit logs, budget history, and population-level artifacts.

- `mpage_orig.py`: the wrapper that runs the original MPaGE baseline under the same budget-accounting and output conventions used by BMAB.
- Experiment harness: suite definitions, run scripts, aggregation, comparison, metric recomputation, diagnostics, and figure-generation code.

This organization separates methodological logic from experiment execution. The core algorithm is implemented in the BMAB orchestration module, while the repeatable experimental matrix is controlled by the experiment configuration and runner.

The separation is important for thesis reproducibility. Methodological claims in Chapter 3 are tied to the algorithm modules, while empirical claims in Chapter 5 are tied to the experiment and analysis modules. This avoids mixing two different concerns: changing the scheduler should happen in the core BMAB implementation, whereas changing which tasks, budgets, or seeds are run should happen in the experiment harness. The implementation mostly follows this separation, which makes it possible to regenerate result tables without rewriting the method itself.

4.2 Software Environment

The dependency specification files list the main Python libraries, including `numpy`, `scipy`, `pymoo`, `matplotlib`, `pandas`, `tqdm`, and an OpenAI-compatible client. The command-line interface uses `gpt-4o-mini` as the default generator and clusterer model unless overridden by CLI arguments. API credentials are read from local secret files; their contents are not part of the thesis and are not required for interpreting the algorithmic design.

The available implementation materials do not specify the exact hardware, GPU, operating system version, API latency, token cost, or monetary cost per call used to produce the reported result set. No information was found in the source code or documentation for these environment details. For this reason, the thesis treats budget primarily as the number of charged LLM calls rather than as wall-clock cost or monetary cost.

4.3 Benchmark Evaluators and Task Preparation

The BMAB implementation imports task evaluators packaged with the experimental framework. Each task defines both a template for the generated heuristic function and an evaluator that executes the generated function inside an archive-based inner search procedure.

The Bi-TSP evaluator uses four fixed instances, problem size 20, a timeout of 60 seconds, an initial archive of 100 random tours, and 2,000 heuristic calls per instance.

The Tri-TSP evaluator uses 20 fixed instances, problem size 20, a timeout of 90 seconds, an initial archive of 100 random tours, and 20,000 heuristic calls per instance.

The Bi-CVRP evaluator uses eight fixed instances, problem size 100, capacity 50 for this size range, a timeout of 90 seconds, 10 initial route sets, and 6,000 heuristic calls per instance.

The Bi-KP evaluator uses eight fixed instances, problem size 200, capacity 25 for this size range, a timeout of 90 seconds, 20 initial feasible item selections, and 8,000 heuristic calls per instance.

The task instance-generation files set the NumPy random seed to 2025 before generating benchmark instances. Therefore, the problem instances are fixed under the current code. The experiment seeds 2025–2029 mainly affect the outer heuristic-generation and selection process rather than creating new benchmark instance sets.

4.4 Score and Population Data Model

The core data object flowing through the system is a generated heuristic function. Conceptually, a function object stores the generated code, metadata, and an optional score. A score is valid only when the code can be executed by the corresponding task evaluator and returns behavior that can be measured. For all benchmark tasks analyzed in this thesis, the outer score has two components:

$$s(h) = \left(-\overline{HV}_{\text{inner}}(h), \overline{\tau}(h)\right). \tag{4.1}$$

The first component is negated because the outer population manager follows a minimization convention. The second component is average runtime. This two-dimensional convention is used even for Tri-TSP, whose inner archive has three objective dimensions. The tri-objective inner archive is first summarized by inner hypervolume, and that scalar is then paired with runtime at the outer level.

The population manager keeps a bounded set of generated heuristics. New children enter a pending population during a generation and are later merged through survivor selection. This detail is significant under a strict budget. If a run stops in the middle of a generation, pending valid children must still be considered before the final population is recorded. The finalized implementation includes this flush behavior, which prevents useful generated heuristics from being lost simply because the budget ended before a complete generation was finished.

Dominance in the outer population is evaluated over heuristic scores, not over source-code similarity. Semantic clustering is used for parent organization and budget allocation, but the final population is determined by numerical objectives. Thus, a semantically novel

heuristic that is slow and low-quality can still be dominated, while a semantically similar heuristic can survive if it improves the quality–runtime trade-off.

4.5 Heuristic Generation and Evaluation Pipeline

Each generated heuristic is represented as Python code containing a task-specific `select_neighbor` function. The LLM is prompted using MPaGE prompt templates, the response is parsed into executable code, and the evaluator attempts to run the function in a controlled process. A successful evaluation returns a two-dimensional outer score: negated inner hypervolume and runtime. If the code fails to parse, raises an exception, violates task constraints, times out, or returns an invalid object, the evaluator may return a null score.

The experimental framework uses the same conceptual evaluation interface across all benchmark families. For each fixed problem instance, the evaluator creates an initial archive, repeatedly calls the generated `select_neighbor` heuristic, inserts feasible and non-dominated candidates, removes dominated archive members, and computes the inner hypervolume of the final archive. The heuristic score used by the outer BMAB population is then averaged over the task instances.

Tri-TSP illustrates the two-level nature of the evaluation. Although the inner task has three distance objectives, the outer heuristic score remains two-dimensional: $(-HV_{\text{inner}}, runtime)$. Accordingly, the outer reference point for Tri-TSP is two-dimensional, `(20.0,60.0)`.

4.6 Execution Flow of a BMAB Run

A single BMAB run proceeds through a fixed sequence of stages. First, the command-line entry point resolves the task evaluator, population size, budget, seed, model names, and reward-mode settings. It then constructs the budget tracker, profiler, population manager, cluster manager, bandit controllers, and reward computer.

Second, the method performs an initialization phase. Initial heuristic programs are generated and evaluated until the initial population is available or the budget is exhausted. These initial candidates provide the first set of scores from which PFG selection and semantic clustering can operate. Initialization is therefore not only a warm-up step; it determines the early structure of the outer search space.

Third, each generation starts by selecting elites and preparing a clustering pool. The implementation retains the MPaGE idea that promising heuristic programs should guide the next prompt context, but the finalized BMAB implementation also expands the clustering pool so that the cluster bandit can reason over a richer set of semantic regions. The cluster

manager either obtains an LLM-based partition or falls back to a deterministic singleton-style partition when the budget is too small to justify a clustering call.

Fourth, the inner generation loop repeatedly selects an operator and a cluster arm. The operator-level bandit chooses between mutation and crossover. The cluster-level bandit chooses which semantic region should receive the current call under the selected operator family. Parent heuristics are then sampled, with higher weight assigned to parents that also appear in the PFG-selected elite set. The LLM generates a child program, the evaluator scores it, and the reward computer converts the outcome into a scalar feedback signal.

Finally, the run writes artifacts. The profiler records budget-curve points, AUBC, budget history, bandit logs, and population snapshots. The finalization step flushes pending offspring into the managed population before final HV is recorded. This flow makes the final result dependent on all valid generated work, not only on completed generations.

4.7 Budget Accounting and Call Semantics

The implementation uses call-based budget accounting. A budget unit corresponds to an LLM call made for generation or clustering. This abstraction is simpler than token-level or monetary accounting, but it has two advantages for the current thesis. First, it is directly observable from the code path: every charged action passes through the budget tracker. Second, it is stable across tasks, which makes the task–budget matrix easier to compare.

The budget tracker enforces two properties. The first is monotonic consumption: once a call is charged, the consumed budget cannot decrease. The second is affordability checking: an action should not be taken if the remaining budget cannot pay for it. The finalized method uses this not only as a safety mechanism but also as part of the algorithm. Near the end of a run, the implementation avoids spending the last available unit on clustering if doing so would leave no budget for generating a heuristic. This design choice reflects the principle that a clustering call has value only if it can guide at least one subsequent generation call.

Call-based budget accounting does not capture every practical cost. It ignores prompt length, completion length, API latency, and evaluator runtime. The implementation artifacts do not provide these measurements for the result set. Nevertheless, call budget is an appropriate first-order abstraction for comparing scheduling policies because it measures the scarce generative action that the proposed method controls.

4.8 Experiment Configuration

The main experimental matrix defines the task set as `bi_tsp`, `tri_tsp`, `bi_cvrp`, and `bi_kp`. The budgets are $B \in \{25, 50, 100, 200\}$, and the seeds are 2025, 2026, 2027, 2028, and 2029.

The function `pop_size_for` selects the managed population size according to budget. For the two-objective tasks, the population sizes are 4, 6, 8, and 10 for budgets 25, 50, 100, and 200, respectively. For Tri-TSP, the configuration adds two extra individuals, yielding population sizes 6, 8, 10, and 12.

The bandit parameters used in the reported runs are $c_{\text{op}} = 1.0$, $c_{\text{cluster}} = 1.0$, $\gamma_{\text{budget}} = 0.5$, prior count 1, and prior reward 0.5. The reward weights are $w_q = 1.0$, $w_d = 0.3$, and $w_r = 0.2$, with invalid-code penalty 1.0 and rank window 50. Page–Hinkley uses $\delta = 0.005$ and threshold 0.5. These values are exposed through the command-line configuration and the corresponding scheduler and reward modules.

The configuration intentionally keeps population size coupled to budget. At small budgets, a large population would consume most of the budget during initialization and leave too few later decisions for the bandit to learn. At large budgets, a larger population can preserve more heuristic diversity and provide richer parent material. Tri-TSP receives a slightly larger population because the inner problem has three objectives and therefore tends to require more diversity to represent useful trade-offs.

4.9 Experiment Suites and Run Scripts

The experiment harness separates suite definition, execution, aggregation, and comparison.

The experiment runner enumerates cells of the form `(ablation,task,budget,seed)`, creates output directories, and dispatches each cell to either the proposed BMAB entry point or the MPaGE-orig wrapper. It skips completed cells unless `--force` is specified. The aggregation code scans result directories and creates aggregate CSV summaries, while the comparison code performs paired comparisons against a selected baseline for a selected metric.

Several shell launchers are provided for common workflows. One launches the standard BMAB ablation grid, one launches the MPaGE-orig versus BMAB comparison, and one launches the reward-mode comparison among Final-HV reward, Dense reward, and Hybrid reward before running aggregation and comparisons for both `hv_final` and AUBC.

The result set analyzed in Chapter 5 combines the complete MPaGE comparison matrix with the complete reward-mode matrix. It therefore covers four thesis-facing setups: Final-HV reward, Dense reward, Hybrid reward, and MPaGE-orig. Other component abla-

tions, such as `no_ph`, `no_diversity`, `op_only`, and `cluster_only`, are available in the codebase but are not analyzed as complete matrices in Chapter 5.

4.10 Generated Artifacts

Each run writes a structured set of artifacts that makes post-hoc analysis possible. The most important artifacts are:

- `samples/`: generated heuristic samples, including code and available scores.
- `population/`: final and intermediate heuristic-population records.
- `budget_curve.json`: a sequence of points containing consumed budget, outer hypervolume, and Pareto population size.
- `budget_history.json`: the budget-consumption history.
- `bandit_log.json`: selected operators, selected cluster arms, rewards, and related bandit diagnostics.
- `aubc.json`: the final AUBC value computed from the budget curve.
- `run_log.txt`: execution logs for the run.

The numerical summaries used in Chapter 5 are materialized in thesis-local table files. This makes the reported values self-contained in the thesis while preserving compatibility with the experiment aggregation scripts when new runs are added.

The artifact structure also supports anomaly inspection. For example, if a cell has positive final HV but zero AUBC, the corresponding `budget_curve.json` and `aubc.json` files can be checked directly. The thesis therefore treats generated artifacts not only as outputs but as audit records that make it possible to validate the aggregation pipeline.

Another important artifact is the saved sample set. Sample files make it possible to inspect the generated code and verify whether a run produced valid heuristics, null-score candidates, or repeated patterns. The current analysis does not perform qualitative code motif analysis, but the presence of these artifacts would support such an extension. This is especially relevant for future work on explaining why particular prompt families or semantic clusters become productive.

4.11 Visualization and Analysis Code

The figures used in Chapter 5 are generated from result CSV and JSON files. The consolidated four-setup overview script creates AUBC plots, final-HV plots, budget curves, pairwise win matrices, normalized-score summaries, mean-rank summaries, and trade-off figures. The thesis includes local copies of the generated figures so that the compiled document is self-contained.

This separation between data, plotting code, and thesis figures is useful for reproducibility. If new runs are added, the same aggregation and visualization pipeline can be executed again to update the numerical summaries and figures without changing the methodological exposition.

The visualization pipeline uses several levels of aggregation. Per-run artifacts are first converted into row-level summaries containing task, budget, seed, setup, AUBC, final HV, valid count, and diagnostic rates. These rows are then aggregated into task–budget cells. Normalized scores are computed within each cell so that tasks with very different absolute hypervolume scales can be compared fairly. Finally, the analysis scripts generate overview plots, pairwise win matrices, and budget-curve figures. This sequence is important because it prevents large-scale tasks such as Tri-TSP from dominating cross-task conclusions simply because their absolute outer-HV values are larger.

The thesis includes figure copies rather than linking directly to the experiment output directory. This is a practical LaTeX decision: it makes the thesis source self-contained and avoids broken figure paths if experiment folders are archived or moved. The original result files remain the source of truth for numerical regeneration, while the thesis-local copies provide a stable compiled document.

4.12 Reproducibility Considerations

The codebase provides several mechanisms for reproducibility: fixed benchmark-instance generation, explicit experiment seeds, deterministic suite definitions, per-run output directories, logged budget curves, and saved generated samples. These mechanisms make it possible to inspect what happened in each run and to recompute aggregate statistics from stored artifacts.

However, exact bit-level reproducibility is not guaranteed. The system depends on an external LLM API, and the implementation artifacts do not provide a complete snapshot of the model backend, sampling behavior, or service-side changes over time. The generated code can also be stochastic, because heuristics may call `random` or `numpy.random`. Therefore, the thesis interprets the reported results as reproducible under the stored artifacts and code paths, while recognizing that re-running the same experiment at a later date may

produce slightly different generated heuristics.

4.13 Implementation Caveats

Three implementation caveats are important when interpreting the results. First, MPaGE-orig and BMAB do not record budget history with identical granularity. BMAB records detailed budget events, whereas the MPaGE wrapper records a more aggregated history. Therefore, the number of budget-history rows should not be compared as if they were identical event types.

Second, BMAB and MPaGE-orig do not store generated samples with identical conventions. BMAB stores accepted or scored heuristic samples in several output locations, while the MPaGE wrapper can retain additional failed samples. For this reason, Chapter 5 avoids using sample-storage diagnostics as primary thesis-facing performance metrics and instead focuses on AUBC and final outer hypervolume.

Third, the runtime timeouts in some benchmark configuration files are not always identical to the timeout values used directly in the Python evaluator classes. When there is such a discrepancy, the thesis prioritizes the evaluator source code because it is the executed path used by the implementation.

Chapter 5

Experiments and Evaluation

5.1 Experimental Questions

This chapter evaluates the proposed budget-aware heuristic-design framework under finite LLM-call budgets. The analysis is based on the consolidated result artifacts generated from 320 runs: four setups, four benchmark problem families, four budget levels, and five random seeds. The four setups consist of three parallel quality-signal variants, namely Final-HV reward, Dense reward, and Hybrid reward, together with MPaGE-orig as the budget-wrapped baseline.

The chapter addresses three experimental questions. First, which setup gives the strongest overall budget-aware performance across tasks and budgets? Second, does better budget efficiency, measured by the area under the budget curve (AUBC), also correspond to better final heuristic-population quality, measured by final outer hypervolume? Third, how do the three quality-signal variants differ when the rest of the scheduling and population-management pipeline is held fixed?

All numerical claims in this chapter are derived from generated result files and summary tables. The chapter does not infer results from undocumented runs. Whenever a quantity is aggregated across tasks, budgets, or seeds, the aggregation rule is stated before the corresponding table or figure.

5.2 Experimental Setups

Table 5.1 defines the four setups compared in this chapter. Each row represents one experimental setup. The first column gives the setup name used throughout the thesis, the second column states its experimental role, and the third column summarizes the configuration. The three reward-based setups are treated as comparable alternatives rather than as a default method plus secondary variants.

Table 5.1: Experimental setups analyzed in the final comparison.

Setup	Purpose	Configuration
Final-HV reward	Quality-signal variant	Uses the shared two-level bandit scheduler, Page–Hinkley drift handling, budget annealing, diversity/rank reward terms, pending-offspring flushing, and a final managed-population HV quality signal.
Dense reward	Quality-signal variant	Uses the same scheduling pipeline but changes the main quality signal to immediate hypervolume-improvement feedback.
Hybrid reward	Quality-signal variant	Uses the same scheduling pipeline but combines dense immediate feedback with final managed-population HV feedback.
MPaGE-orig	Baseline	Runs the original MPaGE workflow under the same measurement protocol so that budget curves, AUBC, and final-HV artifacts can be collected consistently.

The Final-HV reward setup uses the two-level budgeted bandit scheduler, front-aware cluster priors, Page–Hinkley drift handling, final pending-offspring flushing, budget-aware exploration annealing, and a quality signal based on final managed-population HV gain. It tests whether aligning bandit feedback with the terminal managed-population metric improves final heuristic-population quality.

The Dense reward setup uses the same scheduling and population-management pipeline but replaces the final-HV quality signal with immediate hypervolume-improvement feedback. It tests whether denser local improvement signals help the controller make useful decisions earlier in the budget horizon.

The Hybrid reward setup combines dense immediate feedback with final managed-population feedback. Its purpose is to test whether short-horizon local improvement and terminal-population alignment are complementary. MPaGE-orig preserves the original MPaGE workflow but is wrapped in the same budget-recording protocol so that budget curves, AUBC, and final-HV values can be compared consistently.

5.2.1 Benchmark Problems

The experiments use four multi-objective combinatorial optimization benchmarks. Table 5.2 makes the benchmark names explicit before the quantitative results are discussed. Each row corresponds to one benchmark family, explains what a candidate solution represents, identifies the objectives, and reports the evaluator configuration used by the implementation. These tasks are suitable for this thesis because they cover different discrete representations: permutation tours, vehicle-route sets, and binary item-selection vectors.

Table 5.2: Benchmark problem families used in the experimental evaluation.

Benchmark	Full name	Multi-objective formulation	Evaluation configuration
Bi-TSP	Bi-objective Traveling Salesman Problem	A solution is a permutation tour. Each city has two coordinate systems, and the tour length is minimized in both systems.	Four fixed instances with 20 cities. Each heuristic is applied for 2,000 inner calls per instance with a 60-second timeout.
Tri-TSP	Tri-objective Traveling Salesman Problem	A solution is a permutation tour. Each city has three coordinate systems, producing three tour-length objectives to minimize.	Twenty fixed instances with 20 cities. Each heuristic is applied for 20,000 inner calls per instance with a 90-second timeout.
Bi-CVRP	Bi-objective Capacitated Vehicle Routing Problem	A solution is a set of depot-rooted vehicle routes under capacity constraints. The objectives are total travel distance and route makespan.	Eight fixed instances with 100 customers and vehicle capacity 50. Each heuristic is applied for 6,000 inner calls per instance with a 90-second timeout.

Benchmark	Full name	Multi-objective formulation	Evaluation configuration
Bi-KP	Bi-objective Knapsack Problem	A solution is a binary item-selection vector under a capacity constraint. Each item has two profit values, so the objective is to maximize both total profits.	Eight fixed instances with 200 items and capacity 25. Each heuristic is applied for 8,000 inner calls per instance with a 90-second timeout.

Bi-TSP and Tri-TSP evaluate permutation-based local-search heuristics. They differ mainly in objective dimensionality: Bi-TSP optimizes two tour-length objectives, whereas Tri-TSP optimizes three. Bi-CVRP introduces routing feasibility constraints through vehicle capacity, so a generated heuristic must preserve valid route structures while improving distance and makespan. Bi-KP uses a binary representation and a capacity constraint, making it structurally different from the routing benchmarks. Together, these four problems test whether a budget-aware LLM heuristic-design process remains useful across distinct objective spaces and representation constraints.

5.2.2 Hyperparameters and Environment

Table 5.3 summarizes the experimental hyperparameters and local running environment. The table separates what is controlled by the experiment harness, such as budgets and seeds, from what is controlled by the BMAB scheduler, such as population size, operators, reward weights, and bandit parameters. Values that are not explicitly recorded in the implementation are marked with a TODO rather than guessed.

Table 5.3: Experimental hyperparameters and local running environment used for the analyzed result matrix.

Item	Value used in the experiments
Compared setups	Final-HV reward, Dense reward, Hybrid reward, and MPaGE-orig.
Benchmark tasks	Bi-TSP, Tri-TSP, Bi-CVRP, and Bi-KP.
LLM-call budgets	$B \in \{25, 50, 100, 200\}$.
Random seeds	2025, 2026, 2027, 2028, and 2029.

Item	Value used in the experiments
Managed population size	For Bi-TSP, Bi-CVRP, and Bi-KP: 4, 6, 8, and 10 at budgets 25, 50, 100, and 200. For Tri-TSP: 6, 8, 10, and 12 at the same budgets.
Generation target per BMAB generation	The target number of generated offspring is equal to the managed population size.
Initialization limit	The initializer targets one successful population of size `pop_size`; the maximum initialization calls are the larger of 25 and five times `pop_size`.
Variation operators	Two operators are used: mutation and crossover. The operator-level bandit selects between these two operators.
Cluster-level arms	Within a generation, arms are semantic cluster–operator pairs. If the clusterer returns K clusters, the cluster bandit considers up to $2K$ cluster–operator arms.
Selection count	Two parent heuristics are selected when crossover is used.
Budget accounting	Initialization, clustering, generation, and review calls each have unit cost. Review is disabled in the analyzed runs.
Bandit exploration constants	Operator exploration constant $c_{\text{op}} = 1.0$ and cluster exploration constant $c_{\text{cluster}} = 1.0$.
Budget annealing	Enabled for the three BMAB quality-signal variants, with $\gamma_{\text{budget}} = 0.5$.
Reward weights	Quality, diversity, and rank weights are $w_q = 1.0$, $w_d = 0.3$, and $w_r = 0.2$, respectively.
Page–Hinkley settings	Drift threshold $\lambda = 0.5$ and slack $\delta = 0.005$.
Generator LLM	`gpt-4o-mini` through an external OpenAI-compatible chat-completion API.
Cluster LLM	`gpt-4o-mini` through the structured clustering API path.
Generator decoding configuration	Maximum completion length 8,192 tokens, temperature 0.7, streaming disabled, timeout 30 seconds.

Item	Value used in the experiments
Cluster decoding configuration	Structured parse call with a 30-second timeout. TODO: the source code does not explicitly record cluster temperature, top-p, or the exact service-side model snapshot.
Local running environment	Apple M4 chip with 10-core CPU, 8-core GPU, and 16 GB RAM. The local machine runs orchestration and benchmark evaluation; LLM inference is served by the external API.

Each setup is evaluated on the same four tasks, four budgets, and five seeds. Thus, each setup contributes $4 \times 4 \times 5 = 80$ runs, and the complete comparison contains 320 runs. The budget B counts external generation and clustering calls under the call-cost model used by the implementation. This call-based budget is appropriate for the present comparison because the result artifacts do not provide token-level cost, API latency, or monetary cost for each run.

5.3 Metrics and Aggregation

5.3.1 Task–Budget Cells

The basic unit of comparison is the *task–budget cell*. A task–budget cell is an ordered pair

$$c = (t, b), \qquad t \in \mathcal{T},\ b \in \mathcal{B}, \tag{5.1}$$

where $\mathcal{T}$ is the set of four benchmark tasks and $\mathcal{B} = \{25, 50, 100, 200\}$ is the set of budget levels. There are therefore $|\mathcal{T}||\mathcal{B}| = 16$ task–budget cells.

Results are grouped by task and budget for two reasons. First, different tasks have different absolute outer-HV scales, so directly averaging raw values across tasks would overweight high-magnitude tasks. Second, different budgets represent different resource regimes. A setup that performs well at $B = 25$ is not necessarily the same setup that performs best at $B = 200$. Grouping by task and budget preserves this structure before cross-task summaries are computed.

For a setup s, metric m, task t, budget b, and seed r, let $v^{(m)}_{s,t,b,r}$ denote the raw per-run metric value. The cell mean used for comparisons is

$$\bar{v}^{(m)}_{s,t,b} = \frac{1}{|\mathcal{R}|} \sum_{r \in \mathcal{R}} v^{(m)}_{s,t,b,r}, \tag{5.2}$$

where $\mathcal{R} = \{2025, 2026, 2027, 2028, 2029\}$. Thus, table entries and plotted values are generally seed means unless explicitly stated otherwise.

5.3.2 AUBC and Final Outer Hypervolume

The primary metric is AUBC, the area under the outer hypervolume-vs-budget curve. It measures how quickly a setup converts consumed LLM-call budget into heuristic-population quality. Let (x_i, h_i) be the recorded budget curve after sorting by consumed budget, where x_i is consumed budget and h_i is outer hypervolume. The implementation inserts the initial point $(0, 0)$ and uses trapezoidal integration:

$$\mathrm{AUBC} = \frac{1}{B} \sum_{i=1}^{L} \frac{h_{i-1} + h_i}{2} (x_i - x_{i-1}). \tag{5.3}$$

If the last recorded point occurs before the full budget is exhausted, the profiler extends the curve using the last recorded hypervolume. Higher AUBC is better because it indicates that useful heuristic populations are obtained earlier and sustained for more of the budget horizon.

The secondary metric is final outer hypervolume, denoted `hv_final` in the result artifacts. It is extracted from the final `hv` entry of each budget curve. Final outer hypervolume measures the quality of the managed heuristic population at the end of a run. Higher final outer hypervolume is better. It should not be confused with the inner solution-level hypervolume computed inside each benchmark evaluator; Chapter 3 defines this inner–outer distinction in detail.

5.3.3 Normalized Score

Normalized score is used to compare setups across heterogeneous task–budget cells. For AUBC and final HV, the implementation first computes the seed mean $\bar{v}^{(m)}_{s,t,b}$ for each setup in a cell. It then divides by the best mean value observed in the same cell:

$$\mathrm{NS}^{(m)}_{s,t,b} = 100 \frac{\bar{v}^{(m)}_{s,t,b}}{\max_{s' \in \mathcal{S}} \bar{v}^{(m)}_{s',t,b}}, \qquad m \in \{\mathrm{AUBC}, \mathrm{FinalHV}\}. \tag{5.4}$$

Here $\mathcal{S}$ is the set of compared setups. A score of 100 means that the setup has the best mean value in that task–budget cell. A score of 95 means that it reaches 95% of the best observed mean value in that cell. Higher normalized score is better.

This normalization is necessary because raw hypervolume scales are not comparable across tasks. For example, a small raw change on one benchmark may be practically important, while a larger raw change on another benchmark may be minor relative to that task's scale. Cell-wise normalization allows the overall summary to measure relative competitiveness rather than absolute magnitude.

The mean normalized score reported in summary tables is

$$\overline{\mathrm{NS}}^{(m)}_{s} = \frac{1}{|\mathcal{C}|} \sum_{(t,b) \in \mathcal{C}} \mathrm{NS}^{(m)}_{s,t,b}, \qquad |\mathcal{C}| = 16. \tag{5.5}$$

The performance score is the arithmetic mean of the AUBC and final-HV mean normalized scores:

$$\text{Perf}(s) = \frac{\overline{\text{NS}}_s^{(\text{AUBC})} + \overline{\text{NS}}_s^{(\text{FinalHV})}}{2}. \tag{5.6}$$

5.3.4 Mean Rank and Pairwise Wins

Mean rank provides an ordering-based summary. Within each task–budget cell and metric, the setups are sorted by their cell means. Since AUBC and final HV are both higher-is-better metrics, rank 1 is assigned to the largest cell mean. Ties are assigned the average of the tied rank positions, matching the aggregation script. The mean rank of setup s for metric m is

$$\overline{\rho}_s^{(m)} = \frac{1}{|\mathcal{C}|} \sum_{(t,b)\in\mathcal{C}} \rho_{s,t,b}^{(m)}. \tag{5.7}$$

Lower mean rank is better. Normalized score and mean rank should be interpreted together: normalized score measures closeness to the best observed value, while rank measures how consistently a setup is ordered ahead of the others.

Pairwise wins are computed at the task–budget-cell level. For two setups a and b, the pairwise win count for a higher-is-better metric is

$$W_{a,b}^{(m)} = \sum_{(t,b)\in\mathcal{C}} \mathbf{1}\left[\bar{v}_{a,t,b}^{(m)} > \bar{v}_{b,t,b}^{(m)}\right]. \tag{5.8}$$

The maximum possible value is 16. A value of 16 means that setup a has a higher mean value than setup b in every task–budget cell.

5.4 Overall Four-Setup Results

Table 5.4 summarizes the four setups using the normalized-score and mean-rank definitions from Equations 5.4–5.6. Each row is one setup. The AUBC and final-HV score columns report mean normalized scores over 16 task–budget cells. The rank columns report mean ranks over the same cells, where lower is better. The performance score averages the AUBC and final-HV normalized scores.

Table 5.4: Overall normalized scores and mean ranks across the 16 task–budget cells. Scores are normalized within each task–budget cell, where 100 denotes the best setup in that cell. Lower mean rank is better.

Setup	AUBC sc.	AUBC rk.	Final-HV sc.	Final-HV rk.	Perf. sc.
Final-HV reward	96.2	1.88	97.6	1.88	96.9
MPaGE-orig	76.2	3.94	91.2	3.19	83.7
Dense reward	95.4	2.00	96.1	2.50	95.8
Hybrid reward	96.2	2.19	97.6	2.44	96.9

Figure 5.1 visualizes the same two normalized-score columns from Table 5.4. Each bar is the mean normalized score of one setup for either AUBC or final HV. Higher bars indicate stronger relative performance across the full task–budget matrix.

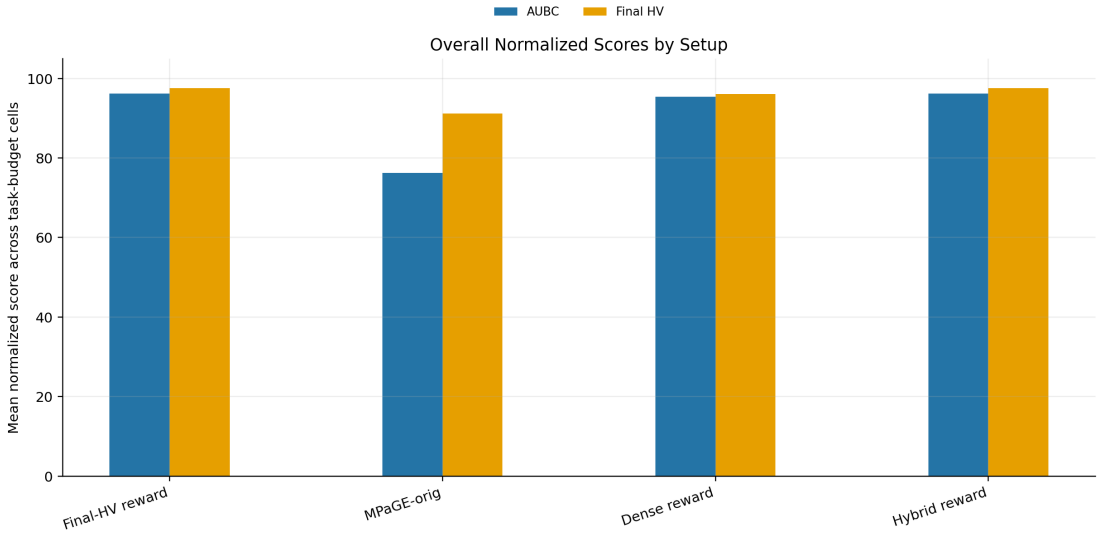


Figure 5.1: Overall normalized scores for AUBC and final outer hypervolume across the four setups.

The strongest overall performance is shared by Final-HV reward and Hybrid reward. Both obtain a performance score of 96.9. Final-HV reward has the best mean final-HV rank, while Hybrid reward has a nearly identical normalized final-HV score and a marginally higher normalized AUBC score. Dense reward is also competitive, with a performance score of 95.8. MPaGE-orig is clearly separated by its lower AUBC score, although its final-HV score is closer to the reward-based variants.

This result should be interpreted carefully. The evidence does not show that one reward design dominates every metric and every task. Instead, it shows that the three budget-aware quality-signal variants form a strong family relative to MPaGE-orig, with small but meaningful differences among the variants. Final-HV reward is the most attractive option

when terminal population quality is the main objective, whereas Hybrid reward is slightly stronger in average normalized AUBC.

5.5 Task-Level Patterns

Table 5.5 breaks the normalized scores down by benchmark family. Each value is computed by normalizing within each task–budget cell and then averaging across the four budgets of that task. Rows are setups and columns are benchmark tasks. The table helps reveal whether the overall result is driven by one benchmark or appears across multiple problem structures.

Table 5.5: Task-level mean normalized scores for AUBC and final outer hypervolume. Scores are normalized within each task–budget cell before averaging across budgets.

(a) AUBC normalized score

Setup	Bi-TSP	Tri-TSP	Bi-CVRP	Bi-KP
Final-HV reward	99.5	96.9	96.4	92.0
MPaGE-orig	88.7	77.5	66.1	72.5
Dense reward	98.3	98.4	95.2	89.7
Hybrid reward	98.1	99.3	94.8	92.7

(b) Final-HV normalized score

Setup	Bi-TSP	Tri-TSP	Bi-CVRP	Bi-KP
Final-HV reward	99.6	100.0	99.2	91.7
MPaGE-orig	99.6	99.8	87.3	78.1
Dense reward	99.1	99.7	94.1	91.5
Hybrid reward	98.9	99.8	93.1	98.5

The Bi-TSP results show that all setups are relatively close in final HV. MPaGE-orig even obtains a strong final-HV normalized score on Bi-TSP, indicating that the original workflow can reach competitive terminal heuristic populations on this benchmark. However, its AUBC score is lower than every reward-based variant. The main Bi-TSP advantage of the budget-aware scheduler is therefore faster conversion of budget into a useful population, not necessarily a much better endpoint.

Tri-TSP provides a clearer budget-trajectory improvement. Final-HV scores are high and close for all setups, but MPaGE-orig has a substantially lower AUBC score. This means that the baseline can eventually approach strong terminal values but tends to do so later. The tri-objective setting makes early coverage of the heuristic-population trade-off surface more important, so adaptive cluster and operator allocation is especially useful.

Bi-CVRP has the largest AUBC gap against MPaGE-orig. The baseline AUBC normalized score is 66.1, while the three reward-based variants are all near or above 94.8. This is practically meaningful because Bi-CVRP includes route-capacity constraints and a route-structure representation. A scheduler that reallocates calls away from unproductive regions can therefore have a larger effect than on simpler permutation tasks.

Bi-KP shows a more mixed reward-mode pattern. Hybrid reward obtains the strongest final-HV normalized score on this task, while Final-HV reward remains competitive in AUBC. This suggests that the binary knapsack representation may benefit from feedback that captures both immediate local improvement and final population contribution. MPaGE-orig is substantially weaker in final HV on Bi-KP, indicating that the proposed scheduling framework improves more than only early progress on this benchmark.

5.6 Pairwise Cell-Level Comparison

Table 5.6 reports pairwise task–budget wins using Equation 5.8. Each entry counts how many of the 16 cells are won by the row setup against the column setup. The diagonal is omitted because a setup is not compared against itself. A larger value means that the row setup is more often better than the column setup for the corresponding metric.

Table 5.6: Pairwise task–budget wins across 16 cells. Each entry gives the number of cells in which the row setup has a higher mean metric value than the column setup.

(a) AUBC wins

Row setup	Final-HV reward	MPaGE-orig	Dense reward	Hybrid reward
Final-HV reward	–	15	9	10
MPaGE-orig	1	–	0	0
Dense reward	7	16	–	9
Hybrid reward	6	16	7	–

(b) Final-HV wins

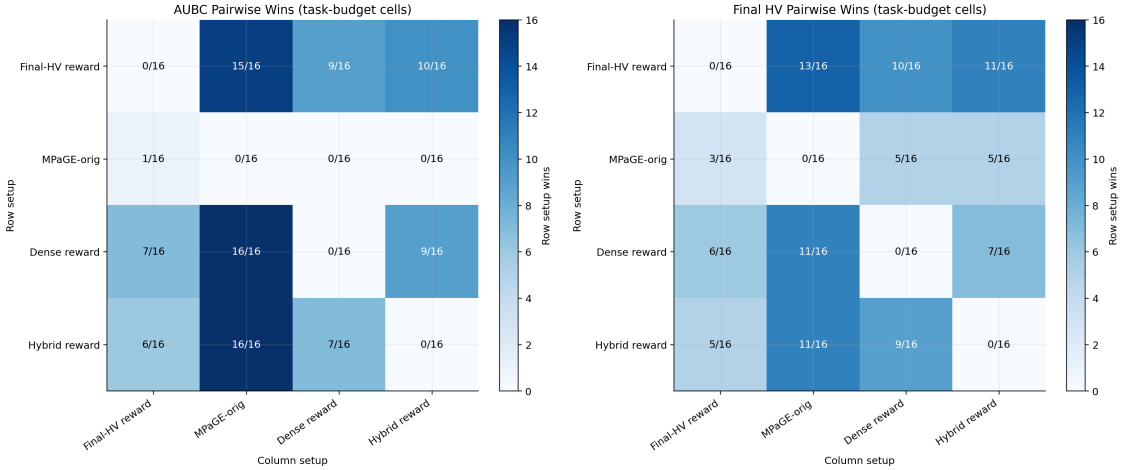
Row setup	Final-HV reward	MPaGE-orig	Dense reward	Hybrid reward
Final-HV reward	–	13	10	11
MPaGE-orig	3	–	5	5
Dense reward	6	11	–	7
Hybrid reward	5	11	9	–

For AUBC, every reward-based setup is clearly stronger than MPaGE-orig. Final-HV reward wins 15 of 16 AUBC cells against MPaGE-orig, while Dense reward and Hybrid

reward each win all 16. This is the strongest evidence that budget-aware scheduling improves budget-integrated behavior relative to the original MPaGE workflow.

Among the three reward-based variants, the AUBC comparison is close. Final-HV reward wins 9 of 16 cells against Dense reward and 10 of 16 against Hybrid reward; Dense reward wins 9 of 16 against Hybrid reward. The final-HV comparison is more favorable to Final-HV reward, which wins 10 of 16 cells against Dense reward and 11 of 16 against Hybrid reward. Thus, the pairwise table supports the same conclusion as the overall table: the framework-level gain over MPaGE-orig is robust, while differences among the three quality signals are more nuanced.

Figure 5.2 visualizes the same pairwise counts as heatmaps. The row setup is the candidate winner, the column setup is the comparator, and darker cells indicate more wins. This figure is useful for quickly seeing asymmetric dominance patterns, especially the strong AUBC separation between the reward-based variants and MPaGE-orig.



(a) AUBC (b) Final HV

Figure 5.2: Pairwise task–budget win matrices for AUBC and final outer hypervolume.

5.7 AUBC Analysis

Figure 5.3 shows the raw mean AUBC values by task and budget. Each subplot corresponds to one benchmark task. The horizontal axis is the LLM-call budget, and each line is one setup. Each plotted point is the mean AUBC over five seeds for the corresponding setup, task, and budget. Higher values are better.

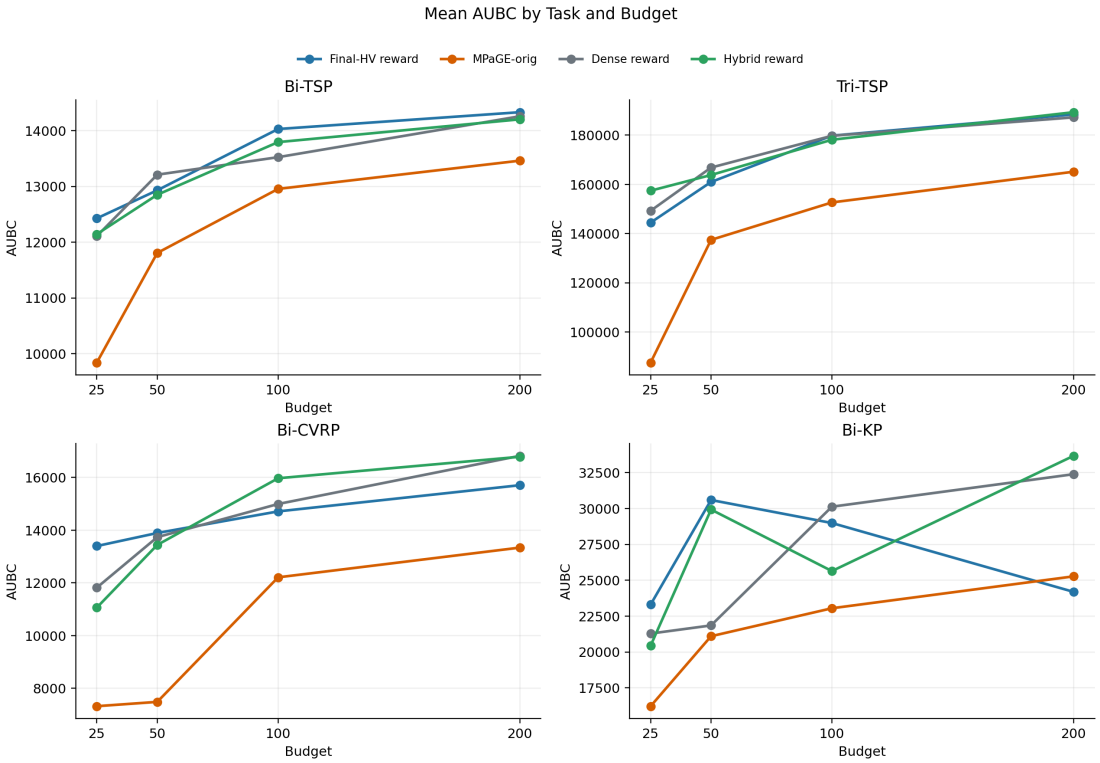


Figure 5.3: Mean AUBC by task and budget for the four experimental setups.

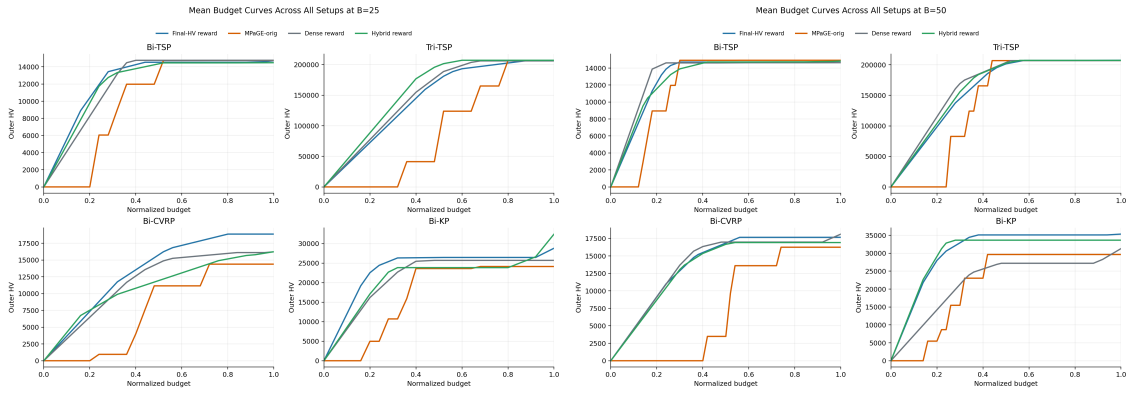
The central pattern is that all three reward-based variants substantially outperform MPaGE-orig in budget-integrated performance. The gap is especially visible in Tri-TSP and Bi-CVRP, where MPaGE-orig tends to improve later in the budget trajectory. Since AUBC integrates the whole curve, delayed improvement is penalized even when final hypervolume eventually becomes competitive.

Final-HV reward obtains the best AUBC value in eight cells and is among the top two in eleven cells. Dense reward obtains four AUBC best cells and twelve top-two cells, while Hybrid reward obtains four AUBC best cells and nine top-two cells. These counts explain why the normalized AUBC scores of the three reward-based variants are close. The reward modes trade off early dense feedback against final-population alignment rather than producing a simple universal winner.

The AUBC result is particularly important at low and medium budgets. When $B = 25$ or $B = 50$, only a small number of useful generation opportunities remain after initialization and clustering overhead. Early scheduling decisions therefore have a large effect on the integral in Equation 5.3. At $B = 100$ and $B = 200$, methods have more opportunities to recover from poor early choices, so final HV can become close even when AUBC remains different. This is why AUBC and final HV should be read together.

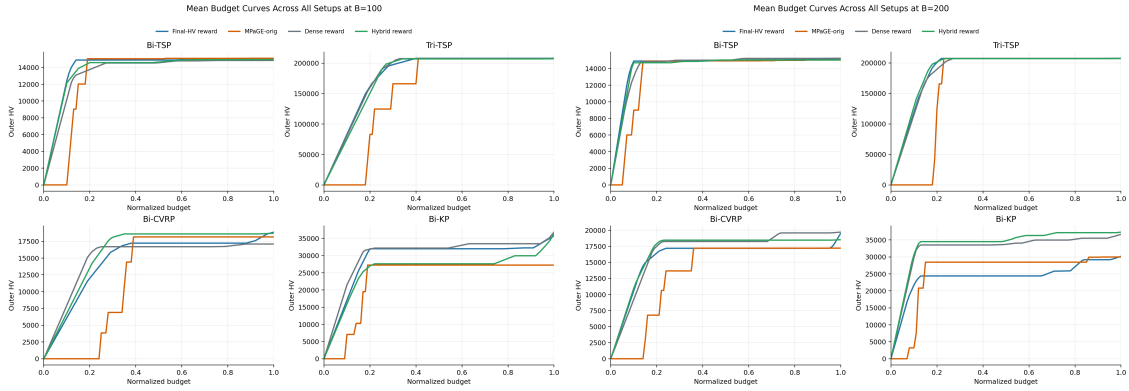
5.8 Budget-Curve Behavior

Figure 5.4 shows mean outer-HV budget curves. Each subfigure fixes a total budget B. Within each subfigure, the four panels correspond to the four benchmark tasks. The horizontal axis is normalized budget, computed as consumed budget divided by total budget. For each setup, per-seed budget curves are first interpolated onto a common grid of 101 normalized-budget points; the plotted line is then the mean outer HV across the five seeds. The vertical axis is raw outer hypervolume, not normalized score.



(a) $B = 25$

(b) $B = 50$



(c) $B = 100$

(d) $B = 200$

Figure 5.4: Mean outer hypervolume curves over normalized budget for the four experimental setups.

The MPaGE-orig curve often looks like a step function: it remains flat over many budget intervals and then jumps when a useful heuristic changes the managed outer population. This behavior follows from the budget wrapper and profiler. Calls are charged for heuristic generation and clustering, but the recorded outer HV changes only when the current heuristic population changes enough to improve its outer hypervolume. Therefore, clustering calls, invalid or non-improving generated heuristics, and generated heuristics that are dominated by the current population can consume budget while leaving the curve unchanged. When a generated heuristic is finally accepted and improves the managed population, the

curve jumps upward.

The three reward-based variants often appear smoother in the seed-averaged plots. This does not mean that their hypervolume changes continuously; individual runs can still contain flat segments. The smoother appearance is produced by two supported mechanisms: the BMAB profiler records population updates at generation boundaries and after final pending-offspring flushing, and the plotted curve averages interpolated curves over five seeds. In addition, the reward-based scheduler can redirect calls during the run based on observed reward feedback, which tends to create more frequent incremental improvements in several cells. The resulting AUBC advantage is therefore a trajectory effect rather than only an endpoint effect.

The y-value at normalized budget 1 corresponds to the mean final outer HV for the same setup, task, and budget when the plotted budget curve and the final-HV summary use the same per-run curve endpoints and the same grouping. It should not be compared directly with an overall normalized-score bar, because normalized-score figures divide by the best setup within each task–budget cell and then average across cells. In short, budget curves show raw seed-mean outer HV within a task, whereas the overall summary shows normalized cross-task competitiveness.

Figure 5.5 excludes MPaGE-orig and zooms the $B = 100$ curves for the three reward-based variants. Each line is again a seed-mean interpolated outer-HV curve. The purpose of the figure is not to introduce a new metric, but to make small trajectory differences among the three reward signals visible after removing the baseline's larger scale differences.

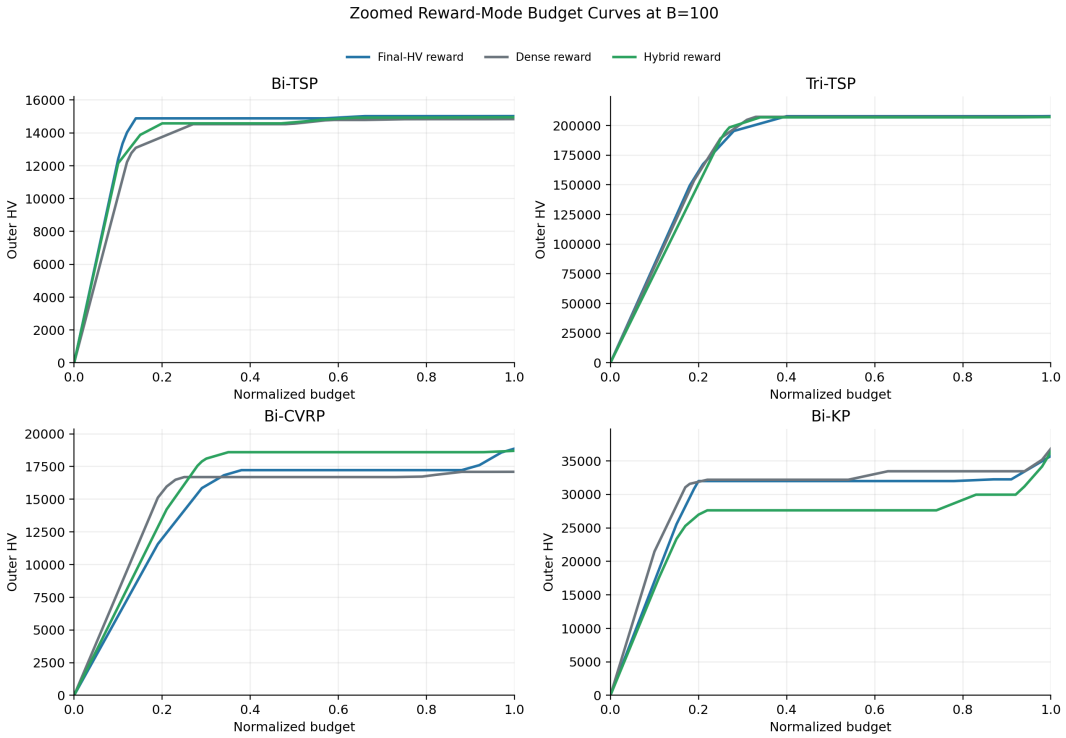


Figure 5.5: Zoomed reward-variant budget curves at $B = 100$, excluding MPaGE-orig to make differences among Final-HV reward, Dense reward, and Hybrid reward more visible.

5.9 Final Hypervolume Analysis

Figure 5.6 shows mean final outer hypervolume by task and budget. Each point is the mean of the final budget-curve endpoint over five seeds. Unlike AUBC, this metric ignores when improvement occurred; it measures only the terminal managed heuristic population after the budget has been consumed. Higher values are better.

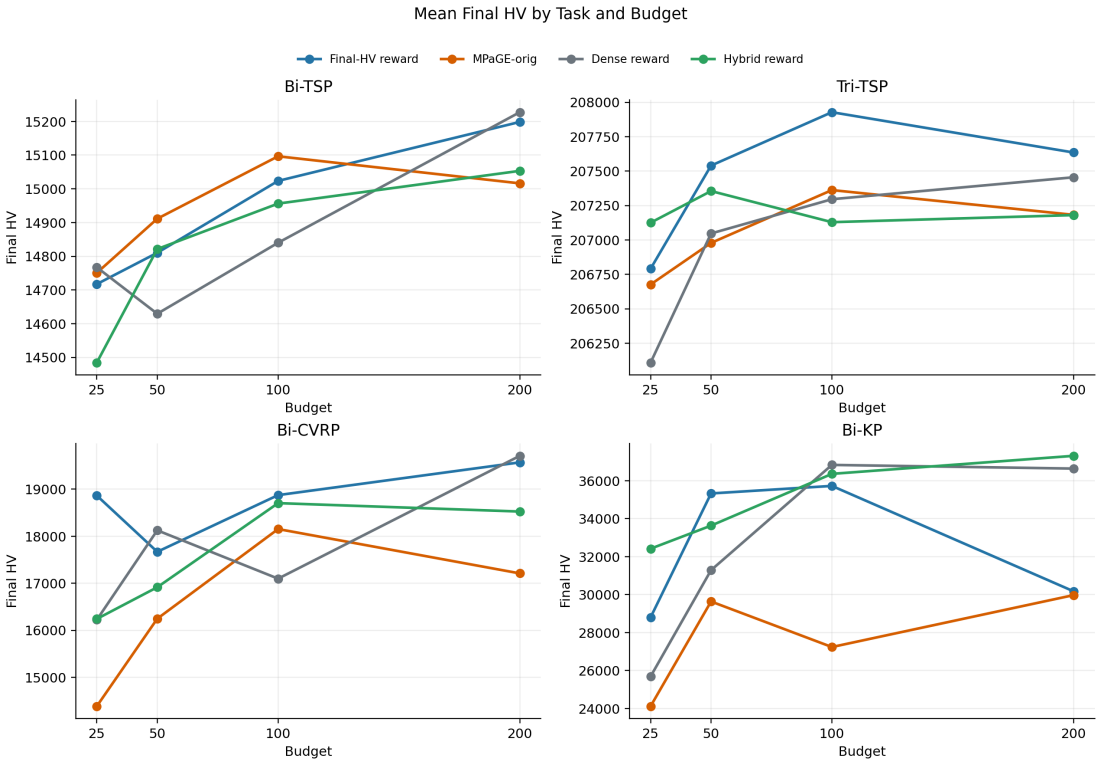


Figure 5.6: Mean final outer hypervolume by task and budget for the four experimental setups.

Final-HV reward has the strongest terminal-quality profile overall. It obtains the best final-HV mean rank, six best final-HV cells, and twelve top-two final-HV cells. It also wins 13 of 16 final-HV cells against MPaGE-orig. This supports the design rationale of using final managed-population HV gain as a quality signal when endpoint population quality is the primary target.

The final-HV results are more mixed than the AUBC results. MPaGE-orig is weak in AUBC but still has two best final-HV cells, and Dense reward and Hybrid reward are competitive in several cells. Therefore, the main advantage of the budget-aware variants is not that they always end with much larger final hypervolume. The more precise conclusion is that they improve the search trajectory and usually maintain or improve terminal population quality.

Figure 5.7 summarizes the relationship between AUBC and final HV. Each point is one setup. The horizontal coordinate is mean normalized AUBC score, and the vertical coordinate is mean normalized final-HV score. A point in the upper-right region indicates that the setup is strong in both budget efficiency and terminal quality.

Figure 5.7: Trade-off between mean normalized AUBC score and mean normalized final-HV score.

Final-HV reward and Hybrid reward occupy the strongest region of the trade-off plot. Dense reward remains close, while MPaGE-orig is separated mainly by its lower AUBC score. This reinforces the practical interpretation: if LLM calls are expensive or runs may be interrupted, AUBC should receive substantial weight; if only the final artifact matters and the complete budget is guaranteed, final HV becomes more important.

5.10 Reward-Mode Interpretation

The three reward-based variants isolate the role of the main quality signal while keeping the finalized implementation otherwise unchanged. Dense reward provides immediate hypervolume-improvement feedback against the current heuristic front. Final-HV reward evaluates the candidate by its effect after managed-population selection, aligning the bandit feedback more directly with the final reported population. Hybrid reward averages these two signals.

The results show that immediate dense feedback remains useful. Dense reward reaches a mean normalized AUBC score of 95.4 and beats MPaGE-orig in all AUBC cells. This suggests that short-horizon improvement information is informative for early progress. However, its final-HV score of 96.1 is lower than Final-HV reward and Hybrid reward, which supports the methodological concern that immediate HVI alone may be less aligned with the capped final population.

Hybrid reward has the highest mean normalized AUBC score by a very small margin and ties Final-HV reward in overall performance score. Nevertheless, it does not surpass Final-HV reward in final-HV rank or pairwise final-HV wins. The two variants therefore represent a practical trade-off: Hybrid reward is slightly stronger in normalized AUBC, whereas Final-HV reward is stronger in terminal outer-HV ranking.

The reward-mode comparison also separates two kinds of improvement. The first is framework-level improvement: budget accounting, adaptive operator and cluster scheduling, front-aware priors, parent weighting, and final pending-offspring flushing make the search process more faithful to the intended outer-population objective. The second is reward-signal choice: the scalar feedback given to the bandit changes which arms are reinforced. Because Final-HV reward, Dense reward, and Hybrid reward share the same scheduling pipeline, their differences should be interpreted as reward-design differences rather than as evidence for entirely separate architectures.

Limitations of the Evidence

The local hardware environment is reported in Table 5.3, but the implementation artifacts do not provide token usage, API latency, monetary cost, GPU utilization, or memory-utilization traces for the reported runs. No information was found in the source code or documentation for these runtime measurements. Consequently, the budget model in this chapter remains call-based.

All reported HV and AUBC values are outer heuristic-population metrics. They are meaningful for comparing heuristic-design processes under the experimental framework used in this thesis, but they are not direct reproductions of the normalized inner-solver metrics reported in the original MPaGE paper.

The experiment matrix compares the three quality-signal variants and MPaGE-orig, but it does not yet fully isolate every architectural component. The codebase contains ablation hooks for Page–Hinkley drift detection, diversity reward, cluster-only control, operator-only control, and budget annealing, but complete result matrices for all of these variants are not part of the current thesis evidence. Therefore, the chapter can conclude that the finalized budget-aware methods work well as a family, but it cannot assign a precise causal contribution to every individual component.

The analysis also remains quantitative rather than qualitative. It does not inspect the generated heuristic code to identify recurring algorithmic motifs. Such an analysis could reveal whether adaptive scheduling discovers genuinely different heuristic families or mainly reallocates budget among similar local variants. The saved samples make this possible, but it is outside the present scope.

Chapter Summary

The result artifacts support three main findings. First, the three reward-based variants are substantially stronger than MPaGE-orig in AUBC, indicating that budget-aware scheduling improves how quickly useful heuristic populations are obtained. Second, Final-HV reward and Hybrid reward form the strongest overall pair: they are tied in performance score, with Final-HV reward leading in final-HV rank and Hybrid reward marginally leading in normalized AUBC. Third, Dense reward remains competitive, showing that immediate HVI-style feedback is useful even though it is less aligned with terminal managed-population quality than Final-HV reward.

Overall, the experiments support the thesis claim that LLM-call allocation is an algorithmically important part of MPaGE-style heuristic design. The evidence suggests that adaptive selection of operators, semantic regions, and quality signals changes the budget trajectory of heuristic-population search while preserving strong terminal quality.

Chapter 6

Conclusion and Future Work

6.1 Summary

This thesis analyzed a budget-aware extension of MPaGE for LLM-based heuristic design in multi-objective combinatorial optimization. The proposed framework preserves MPaGE's core generative and evaluative mechanisms, including SEMO-style archive updates, Pareto Front Grid selection, semantic clustering, prompt-based heuristic generation, and benchmark evaluation. It extends this foundation with an adaptive scheduling layer that treats LLM calls as a scarce resource.

The proposed BMAB-LLM method can be summarized as follows. Each generator or clusterer invocation is charged by a hard `BudgetTracker`. An operator-level UCB controller chooses between mutation and crossover. A cluster-level budgeted UCB controller selects semantic cluster–operator arms within each generation. Page–Hinkley drift detection resets stale cluster statistics when reward behavior changes. A bounded reward function converts heuristic-evaluation outcomes into bandit feedback using final managed-population hypervolume gain, diversity gain, rank smoothing, and invalid-code penalties. The profiler records budget curves, AUBC, bandit logs, and final population artifacts for analysis.

At a broader level, this study shows that MPaGE-style heuristic design can be strengthened by making LLM-call allocation adaptive. MPaGE already provides a productive foundation through PFG selection, semantic clustering, and prompt-based variation. The contribution of the proposed framework is to place these mechanisms under a controller that learns which operators and semantic regions deserve additional budget during the run.

6.2 Main Contributions

The first contribution is the formulation of LLM-based heuristic design as a budget-constrained sequential decision problem. Rather than treating the budget merely as a

stopping condition, this thesis makes budget consumption part of the algorithmic state and evaluates progress over the entire budget trajectory through AUBC.

The second contribution is a two-level bandit scheduler for MPaGE-style heuristic generation. The operator bandit learns long-term preferences between mutation and crossover, while the cluster bandit adapts to the short-lived semantic clusters constructed in each generation. This structure is well matched to the thesis setting: operators are stable across the run, whereas cluster identities are local and non-stationary.

The third contribution is the integration of non-stationarity handling through Page–Hinkley drift detection. Because a semantic cluster can become less useful after repeated exploitation, resetting stale cluster statistics is a practical mechanism for avoiding persistent overconfidence in outdated reward estimates.

The fourth contribution is the reward and population-handling design used by the three quality-signal variants. The implementation rewards the actual generated child, supports Final-HV, Dense, and Hybrid quality signals, prevents the last affordable call from being spent only on clustering, incorporates front-aware cluster priors, weights parents by PFG membership, and flushes pending valid offspring into the final population before result writing.

The fifth contribution is an experimental and analysis pipeline that records not only terminal results but also the process by which budget is consumed. Artifacts such as `budget_curve.json`, `aubc.json`, `bandit_log.json`, and generated thesis figures support analysis of both final quality and budget efficiency.

6.3 Main Findings

The current result set provides strong practical evidence that BMAB-style scheduling improves budget efficiency over the original MPaGE workflow. Across the four-setup matrix, Final-HV reward wins 15 of 16 AUBC task–budget cells against MPaGE-orig, while Dense reward and Hybrid reward each win all 16 AUBC cells against MPaGE-orig. The largest qualitative differences appear in tasks where the baseline budget curve rises later, particularly Tri-TSP, Bi-CVRP, and Bi-KP.

The evidence for final hypervolume is also favorable to Final-HV reward, but it remains more nuanced than AUBC. This setup has the best mean final-HV rank, six best final-HV cells, and twelve top-two final-HV cells. It also wins 13 of 16 final-HV cells against MPaGE-orig. Nevertheless, Dense reward, Hybrid reward, and occasionally MPaGE-orig remain competitive in some terminal cells. Therefore, the correct conclusion is not uniform terminal domination, but improved budget trajectory with generally strong terminal quality.

The reward-mode comparison clarifies an important trade-off among the three

quality-signal variants. Hybrid reward obtains a slightly higher normalized AUBC score than Final-HV reward, whereas Final-HV reward has the strongest final-HV rank. Dense reward remains competitive but is weaker than Final-HV reward in final-HV score. These results show that the three quality signals emphasize different practical objectives: dense feedback supports early local improvement, hybrid feedback balances early and terminal signals, and final-HV feedback supports terminal population quality.

The task-level analysis gives a more nuanced view. On Bi-TSP, several methods reach similar final-HV values, so the BMAB advantage is expressed mainly through better budget trajectory. On Tri-TSP, final-HV values are also close, but BMAB-style methods reach high outer HV earlier. On Bi-CVRP, the AUBC gap against MPaGE-orig is especially large, suggesting that adaptive scheduling is valuable when route feasibility and structured local moves make generation more fragile. On Bi-KP, hybrid reward is particularly competitive, which suggests that dense local feedback may be useful for binary item-selection representations.

The most accurate empirical conclusion is therefore careful rather than absolute. The proposed method should not be described as uniformly dominating every setup on every metric. It should be described as consistently improving budget-integrated performance while maintaining strong terminal quality. This interpretation matches the research objective more closely than a final-HV-only summary would.

6.4 Limitations

The current complete result matrix includes the MPaGE baseline and the three BMAB quality-signal variants. However, it does not include complete matrices for all component ablations. The codebase implements additional variants, including `no_budget_anneal`, `no_ph`, `no_diversity`, `op_only`, and `cluster_only`. Without complete result matrices for these variants, the thesis cannot isolate the causal contribution of every scheduler component.

The local hardware environment is known, but the implementation artifacts do not provide GPU-utilization traces, memory-utilization traces, API latency, token-level cost, or monetary cost for the reported runs. No information was found in the source code or documentation for these runtime measurements. Consequently, the current budget model counts LLM calls rather than modeling heterogeneous token costs or real-time latency.

Finally, the reported HV and AUBC values are outer heuristic-population metrics. They are meaningful for comparing budget-aware heuristic-design processes under the experimental framework used in this thesis, but they are not a direct reproduction of all inner-task solution-quality tables from the original MPaGE paper.

The thesis is also limited by the absence of qualitative code analysis. The saved generated samples make it possible to inspect recurring heuristic motifs, invalid-generation patterns, or semantic differences between clusters, but the current study focuses on quantitative outer-population metrics. Such qualitative analysis would be useful for explaining why particular reward modes or task families behave differently.

6.5 Future Work

The most immediate future direction is to run a complete component ablation matrix. The reward-mode comparison is now available for Final-HV reward, Dense reward, and Hybrid reward, but the effects of Page–Hinkley drift detection, diversity reward, operator-only scheduling, cluster-only scheduling, and budget annealing still require complete controlled sweeps.

A second direction is to extend the budget model from uniform call cost to token- or latency-aware cost. In practical LLM deployments, prompts and completions can differ substantially in length, and different models can have different price and latency profiles. A token-aware budgeted bandit would better reflect deployment constraints.

A third direction is to log and analyze invalid-generation failures more precisely. Separating syntax errors, missing function definitions, runtime exceptions, infeasible outputs, and timeouts would support targeted prompt engineering and repair strategies. Such diagnostics could also clarify whether validity improvements or reward allocation contribute more to AUBC gains.

A fourth direction is to extend the search controller with higher-level planning mechanisms, such as prompt repair, memory over successful heuristic motifs, or structured exploration over operator sequences. These extensions should be evaluated carefully because any additional LLM calls must justify their cost under the same AUBC-oriented budget criterion.

A fifth direction is adaptive reward scheduling. The current results show that Dense reward is useful for early progress and that Final-HV reward is strongest for terminal quality. This suggests a natural schedule in which the controller uses more Dense or Hybrid feedback early and gradually shifts toward Final-HV feedback as the remaining budget decreases.

A sixth direction is qualitative heuristic analysis. Since the implementation stores generated samples, future work can cluster or manually inspect the generated code to identify recurring local-search structures. This could answer questions that pure HV metrics cannot answer, such as whether successful heuristics share common move operators, whether invalid code arises from specific prompt patterns, or whether crossover actually combines useful motifs from two parents.

6.6 Final Remarks

This thesis demonstrates that budget allocation is a substantive algorithmic issue in LLM-driven heuristic design. By adding a two-level bandit scheduler, hard budget accounting, drift detection, and budget-curve profiling to MPaGE, the proposed framework turns a fixed-budget search loop into an adaptive resource-allocation process. The current experiments indicate that this design substantially improves budget-integrated performance, especially on more complex benchmark families. Although further ablations would strengthen the causal analysis, this thesis provides a coherent foundation for studying Generative AI heuristic design under realistic cost constraints.

Bibliography

[1] M. H. Ha, H. Phan, T. D. Doan, T. Dao, D. Tran, and H. T. T. Binh, "Pareto-grid-guided large language models for fast and high-quality heuristics design in multi-objective combinatorial optimization," 2026.

[2] K. Deb, A. Pratap, S. Agarwal, and T. Meyarivan, "A fast and elitist multiobjective genetic algorithm: Nsga-ii," *IEEE Transactions on Evolutionary Computation*, vol. 6, no. 2, pp. 182–197, 2002.

[3] Q. Zhang and H. Li, "Moea/d: A multiobjective evolutionary algorithm based on decomposition," *IEEE Transactions on Evolutionary Computation*, vol. 11, no. 6, pp. 712–731, 2007.

[4] M. Laumanns, L. Thiele, and E. Zitzler, "Running time analysis of multiobjective evolutionary algorithms on pseudo-boolean functions," *IEEE Transactions on Evolutionary Computation*, vol. 8, no. 2, pp. 170–182, 2004.

[5] E. Zitzler and L. Thiele, "Multiobjective evolutionary algorithms: A comparative case study and the strength pareto approach," *IEEE Transactions on Evolutionary Computation*, vol. 3, no. 4, pp. 257–271, 1999.

[6] C. M. Fonseca, L. Paquete, and M. López-Ibáñez, "An improved dimension-sweep algorithm for the hypervolume indicator," in *IEEE Congress on Evolutionary Computation*, 2006, pp. 1157–1163.

[7] J. Blank and K. Deb, "pymoo: Multi-objective optimization in python," *IEEE Access*, vol. 8, pp. 89 497–89 509, 2020.

[8] B. Romera-Paredes, M. Barekatain, A. Novikov, M. Balog, M. P. Kumar, E. Dupont, F. J. R. Ruiz, J. S. Ellenberg, P. Wang, O. Fawzi, P. Kohli, and A. Fawzi, "Mathematical discoveries from program search with large language models," *Nature*, vol. 625, pp. 468–475, 2024.

[9] Á. Fialho, L. Da Costa, M. Schoenauer, and M. Sebag, "Extreme value based adaptive operator selection," in *Parallel Problem Solving from Nature*, 2008, pp. 175–184.

[10] P. Auer, N. Cesa-Bianchi, and P. Fischer, “Finite-time analysis of the multiarmed bandit problem,” *Machine Learning*, vol. 47, pp. 235–256, 2002.

[11] S. Bubeck and N. Cesa-Bianchi, “Regret analysis of stochastic and nonstochastic multi-armed bandit problems,” *Foundations and Trends in Machine Learning*, vol. 5, no. 1, pp. 1–122, 2012.

[12] W. Ding, T. Qin, X.-D. Zhang, and T.-Y. Liu, “Multi-armed bandit with budget constraint and variable costs,” in *Proceedings of the AAAI Conference on Artificial Intelligence*, 2013.

[13] E. S. Page, “Continuous inspection schemes,” *Biometrika*, vol. 41, no. 1/2, pp. 100–115, 1954.